



MSc Cyber Security Engineering

Applied Cyptography

Technical Report

Submitted By:
Madhuchandra Sethunath
Msc Cyber Security Engineering

I hereby declare that this assessment is my own work and that all sources have been acknowledged.

2025-26

Table of Contents

1	<i>System Design and Justification</i>	4
1.1	System Overview	4
1.2	Architectural Design and Key Hierarchy	4
1.3	Cryptographic Control Selection	5
1.4	STRIDE Threat Model	6
1.5	Data Protection and Compliance Considerations	7
1.6	Human Factors and Usability	7
1.7	Design Evolution	7
2	<i>Implementation</i>	8
2.1	Authentication and Credential Security	8
2.2	Role-Based Access Control	10
2.3	File Encryption and Signing	10
2.4	Key Management	11
2.5	Audit Logging	12
2.6	Admin Functions	13
3	<i>Testing and Legal Compliance</i>	14
3.1	Test Accounts	14
3.2	Functional Feature Tests	14
3.3	Access Control and RBAC Tests	16
3.4	Data Integrity Tests	19
3.5	Legal Compliance Mapping	20
4	<i>Reflection</i>	21
5	<i>References</i>	22
6	<i>Appendices</i>	23
	Appendix A	23
	Appendix B	24
	B.1 Authentication and Credential Security	24
	B.1.1 Password Validation and Registration	24
	B.1.2 Multi-Factor Authentication and Session Lockout	25
	B.2 Role-Based Access Control	26
	B.2.1 Researcher Operations	27

B.2.2 Clinician Operations	32
B.2.3 Auditor Operations.....	39
B.3 File Encryption and Signing	42
B.3.1 Symmetric Encryption and Signatures	42
B.3.2 Key Agreement and Wrapping	43
B.3.3 Serialisation Utilities.....	45
B.4 Key Management	46
B.4.1 KEK Derivation and Bundle Creation.....	46
B.4.2 Key Versioning and Rotation	49
B.4.3 Credential-Driven Key Rewrapping	51
B.5 Audit Logging	52
B.6 Storage and Configuration	54
B.6.1 System Constants	54
B.6.2 Database Initialisation and Schemas	55
B.6.3 User and Key Entity Persistence	57
B.6.4 File, Share, and Audit Persistence.....	62
B.7 Main Application Flow and Administrator Functions.....	64
B.7.1 Administrator Controls and System Backup	64
B.7.2 Application Entry Point and Setup Bootstrapping	67

1 System Design and Justification

1.1 System Overview

Aurelia is a secure cross-border clinical research collaboration platform for controlled exchange of sensitive medical information under GDPR Article 9. It supports four roles: Researchers, Clinicians, Auditors, and Administrators. Role separation is enforced through application-level menus and cryptographic scoping, with auditors restricted to verification-only functions and no access to file-decryption keys. The architecture separates the platform core from persistent storage, so plaintext clinical data is never stored.

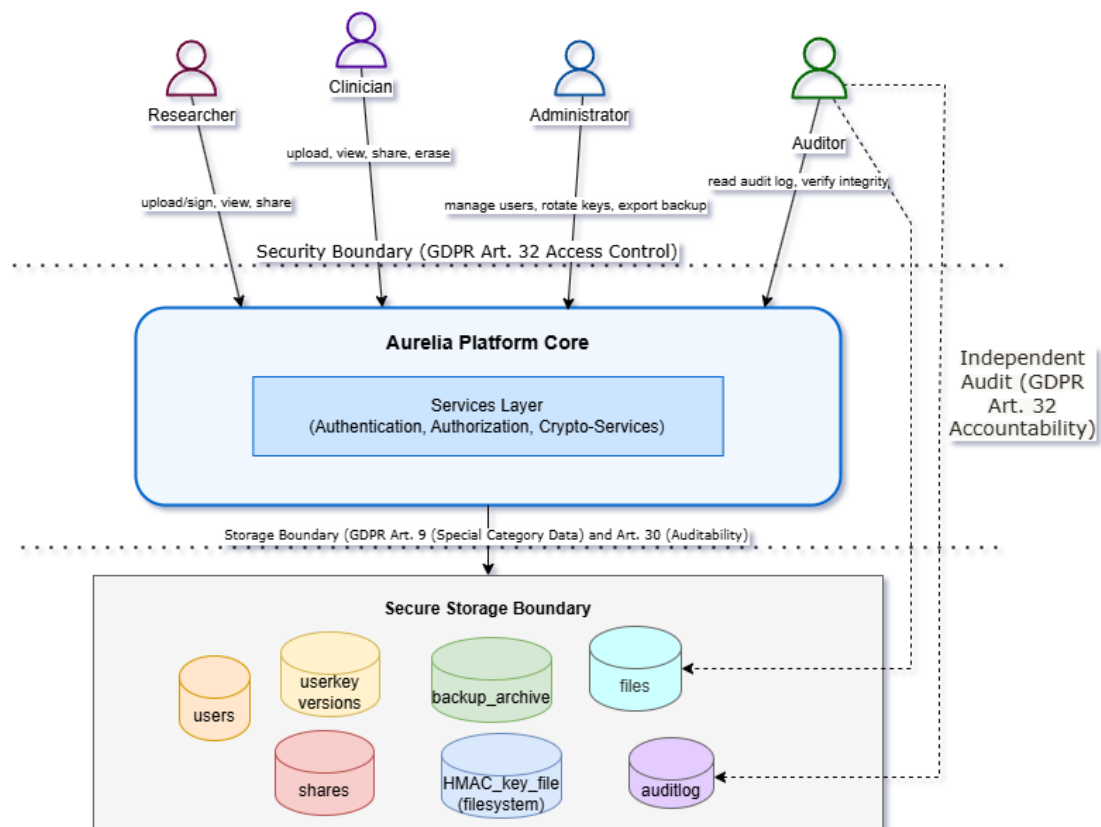


Figure 1.1.1: System architecture and role interactions showing the separation between role-constrained operations and encrypted storage.

1.2 Architectural Design and Key Hierarchy

Aurelia utilizes a per-user hierarchical key model to ensure cryptographic isolation. During registration, `create_initial_keys()` provisions a unique cryptographic identity consisting of one Ed25519 signing keypair and one X25519 key agreement keypair. Private keys are never stored in plaintext; instead, they are encrypted using AES-GCM under a Key Encryption Key (KEK). This KEK is derived from the user's password via Argon2id with a fresh per-version salt. This decouples credential security from identity: when a user changes their password, `unwrap_all_user_private_keys()` updates the wrapping layer without replacing the underlying keypairs or re-encrypting stored files. Key versioning preserves retired versions for historical

verification while keeping the active key state current. The detailed cryptographic and data lifecycle flow is illustrated in Diagram 2.

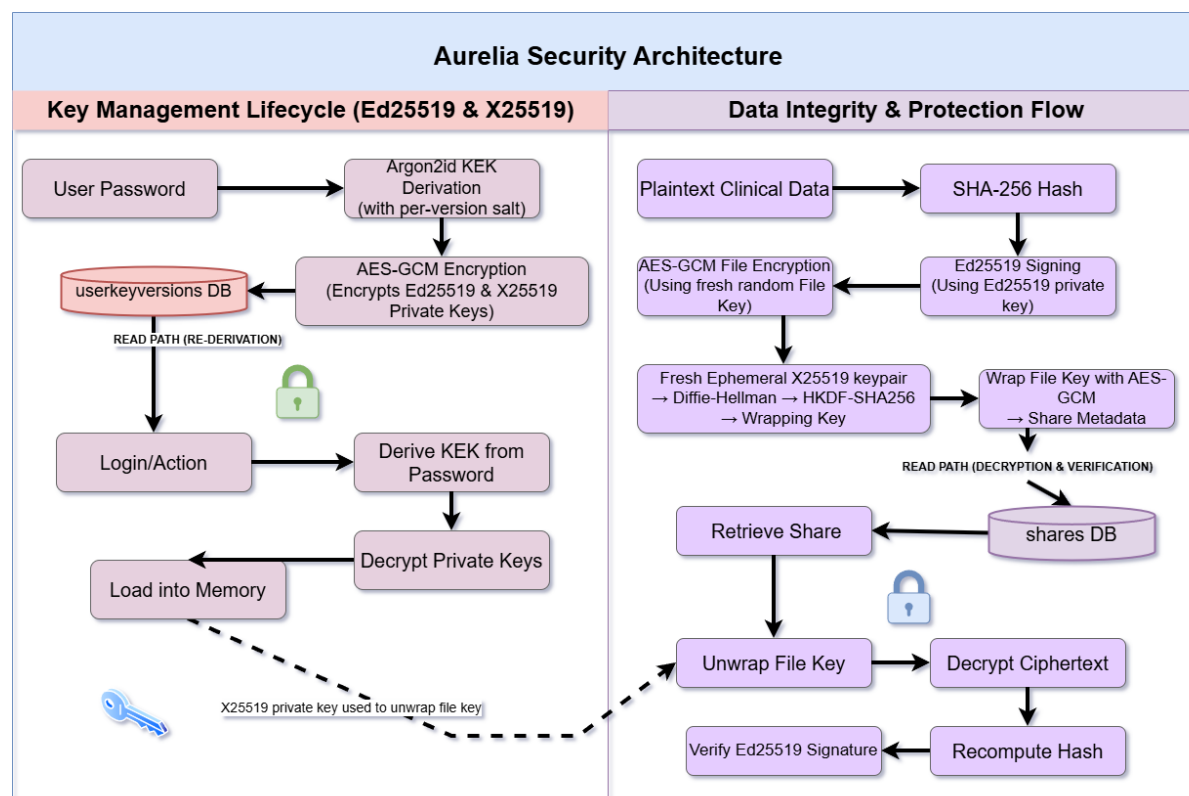


Figure 1.2.1: Key lifecycle and file protection flow, detailing the separation between key-wrapping layers and data-integrity

The file encryption and sharing workflow is detailed in Section 2.

1.3 Cryptographic Control Selection

Primitive	Alternative considered	Rationale for selection
AES-GCM (Dworkin, 2007; OWASP Foundation, 2022)	AES-128-GCM	Authenticated encryption with a stronger 256-bit margin for long-term protection of special-category clinical data; 128-bit security is already infeasible to break in practice, but 256-bit supports defence in depth.
Ed25519 (Josefsson and Liusvaara, 2017)	ECDSA P-256	Avoids the nonce-generation fragility of ECDSA, where nonce reuse can expose private keys; offers compact, deterministic signatures.

X25519 (Langley et al., 2016)	RSA key transport	Supports ephemeral key agreement (Perfect Forward Secrecy) and is better aligned with modern elliptic-curve standards.
Argon2id (Biryukov et al., 2021)	bcrypt / PBKDF2	Memory-hard design provides superior resistance to GPU-accelerated brute-force attacks on password hashes and KEKs.
Chained HMAC	Append-only log	HMAC-chaining binds each record to its predecessor, making retroactive modification or deletion immediately detectable.
SHA-256	SHA-3	Produces a collision-resistant digest suitable for file integrity checks and as input to Ed25519 signing; it is NIST-approved and recommended by NIST for interoperable hash-based applications.

Table 1.3.1: Rationale for selected cryptographic primitives and evaluated alternatives (supporting GDPR Article 32).

1.4 STRIDE Threat Model

Threat (Microsoft STRIDE Team, 2009)	Defensive mechanism	Implementation
Spoofing	MFA, Argon2, Account Lockout	Access requires valid password and TOTP; <code>validate_password_strength()</code> enforces complexity.
Tampering	Auth-encryption, Signatures	Data is decrypted only after verification against stored hashes and Ed25519 signatures.
Repudiation	Chained Audit Logging	HMAC-linking allows <code>verify_log_integrity()</code> to detect alteration of historical records.
Information Disclosure	Encrypted storage, Scope limiting	Content is stored as ciphertext; sharing is limited to recipients via wrapped file keys.
DoS	Throttling, Session expiry	Failed logins trigger temporary account lockouts; sessions are terminated after inactivity.

Elevation of Privilege	RBAC, Administrative separation	Role-based menus restrict access; Auditor functions are logically separated from decryption paths.
-------------------------------	---------------------------------	--

Table 1.4.1: Analysis of potential threats and corresponding mitigation strategies.

1.5 Data Protection and Compliance Considerations

Aurelia is designed to align with GDPR requirements regarding the processing of special-category clinical data **(Article 9) (European Parliament and Council, 2016; ICO, 2024)**.

- Article 9(2)(a) is represented by the consent_basis field, with access constrained through encrypted storage and role-scoped paths.
- Article 17 is enforced by delete_file(), which removes the encrypted dataset and associated sharing records when consent is withdrawn.
- Article 32 is addressed through AES-GCM encryption, per-user key wrapping, password hardening, and MFA, supporting confidentiality and integrity.
- Article 35 is supported by end-to-end encryption, role-scoped access, and tamper-evident audit logging; Article 5(2) is strengthened by the HMAC-chained audit log; Article 5(1)(c) is supported by sanitised audit records that exclude raw patient identifiers.

Legal compliance and functional testing are further detailed in Section 3.

1.6 Human Factors and Usability

The platform is designed to make security controls understandable without making them weak. Role-specific menus separate Researchers, Clinicians, Auditors, and Administrators, reducing confusion and accidental misuse. MFA is presented as a simple second step rather than a complex setup burden, and the login flow uses a clear TOTP prompt so users understand that both password and token are required.

Error handling is intentionally non-specific to avoid username enumeration and factor leakage. This is a deliberate security-usability trade-off: the same vague failure message improves usability consistency while also supporting GDPR Article 32 by reducing information disclosure during authentication. Login and lockout feedback remains clear without revealing sensitive state. Session timeout is also handled in a predictable way: users are returned to the login prompt after inactivity, which makes the security boundary visible rather than surprising. Overall, these usability choices balance clarity, security, and low cognitive load for non-technical clinical users.

1.7 Design Evolution

The design evolved from initial shared-access models, which posed a catastrophic risk if one participant were compromised, to a more granular, per-user identity system.

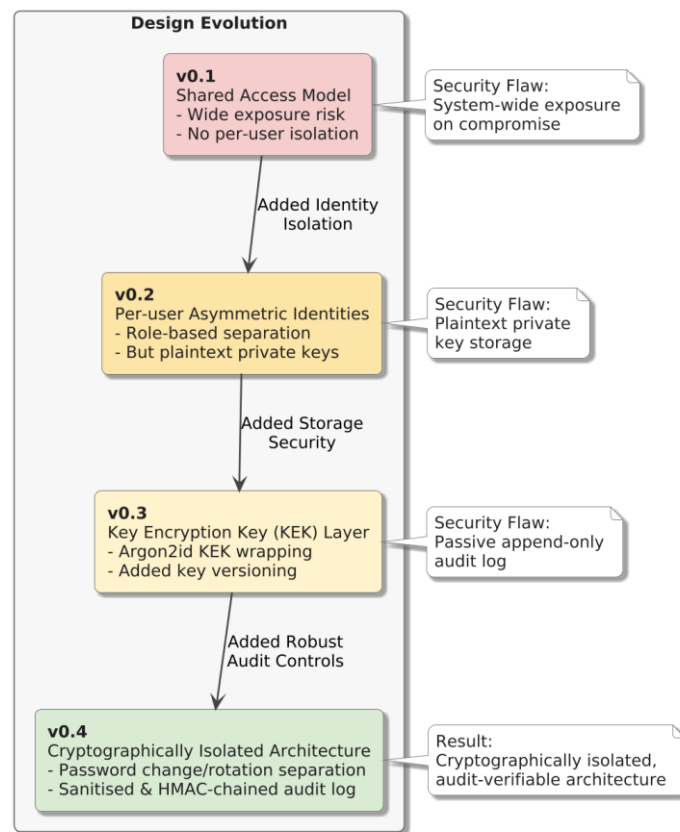


Figure 1.7.1: Design evolution of the Aurelia platform across four iterations, showing the progression from a shared-access model to an architecture with per-user cryptographic isolation, password-derived key protection, and tamper-evident audit logging.

2 Implementation

This section documents the technical realization of the Aurelia platform, mapping functional requirements to the developed Python modules.

2.1 Authentication and Credential Security

Authentication is centralized within `auth.py`, utilizing `register_user()` and `authenticate()` to enforce platform access. Passwords use Argon2id with configurable cost parameters to resist offline cracking. During login, `verify_totp()` checks pyotp MFA tokens, and account lockout reduces username enumeration and credential stuffing risk.


```

[SETUP] No admin account found. Create the built-in admin now.
Set admin password :
Confirm admin password :
[OK] Built-in admin account created.
[!] Admin TOTP secret: 5VA7ZH26Z6BVXB50P2HZORHHKNZSGI64
=====
Aurelia Clinical Research Platform
Cross-Border Clinical Research Collaboration
=====

1 Login
2 Register new account
3 Exit
Option: 

```

Figure 2.1.1: Initial admin account bootstrapping. Demonstrates the one-time system setup flow, ensuring the platform starts with a secure administrative foundation.

```

1 Login
2 Register new account
3 Exit
Option: 2

--- Register New Account ---
First name      : Lily
Last name       : Mathew
Location        : Manchester, UK
Username        : lily_mathew
Select role:
1 Researcher
2 Clinician
3 Auditor
Role: 1
Password requirements: at least 8 characters, one uppercase letter, one lowercase letter, one digit, and one special character.
Password        :
Confirm password :
[OK] User created: lily_mathew (researcher)
[!] TOTP Secret: LYMVUJFTZ44EMZ7GB4HJGJAJ7JUJ52N7
[!] Add to authenticator app:
otpauth://totp/ClinicalResearchPlatform:lily_mathew?secret=LYMVUJFTZ44EMZ7GB4HJGJAJ7JUJ52N7&issuer=ClinicalResearchPlatform

```

Figure 2.1.2: Registration and MFA provisioning. Shows the enforcement of password complexity requirements and the generation of a TOTP secret for secure MFA.

```

1 Login
2 Register new account
3 Exit
Option: 1
Username : lily_mathew
Password :
[DEMO] Current MFA code for lily_mathew: 870360
Authenticator code : 870360

[+] Welcome, Lily Mathew
Role      : researcher
Location  : Manchester, UK

```

Figure 2.1.3: MFA-protected login. Demonstrates the two-step verification process, requiring both primary credentials and a time-based TOTP token.

2.2 Role-Based Access Control

RBAC is enforced in `main.py` through role-specific menu functions (`researcher_menu()`, `clinician_menu()`, `auditor_menu()`, `admin_menu()`). User roles are established at registration and stored in the database. Access to functions is restricted by validating the session role against the user's account attributes.

<pre>[+] Welcome, System Administrator Role : admin Location : Internal --- Admin Menu --- 1 List users 2 Create new user 3 Disable user 4 Enable user 5 Export system backup 6 Change password 7 Logout</pre>	<pre>[+] Welcome, Lily Mathew Role : researcher Location : Manchester, UK --- Researcher Menu --- 1 Upload and sign a finding 2 View my findings 3 View findings shared with me 4 Share a finding with another researcher 5 Change password 6 Rotate my cryptographic keys 7 Logout</pre>
<pre>[+] Welcome, Clara Leon Role : auditor Location : Berlin, Germany --- Auditor Menu --- 1 View audit log 2 Verify audit log integrity 3 Verify a signature 4 Change password 5 Logout</pre>	<pre>[+] Welcome, Jake Thomas Role : clinician Location : Georgia, US 1 Upload patient dataset 2 View my datasets 3 View datasets shared with me 4 Share a dataset with another clinician 5 Erase a dataset 6 Change password 7 Rotate my cryptographic keys 8 Logout</pre>

Figure 2.2.1: Role-restricted system menus. Composite view showing how functional access is dynamically constrained based on the authenticated user's role (Researcher, Clinician, Auditor, Administrator).

2.3 File Encryption and Signing

Data integrity and confidentiality are managed by `crypto_core.py`. When a user uploads a finding or dataset, `main.py` computes a SHA-256 hash, signs the digest with the user's Ed25519 private key, and encrypts the content using `encrypt_file()`. This workflow ensures that file authenticity can be independently verified by auditors without requiring decryption.

```

--- Researcher Menu ---
1 Upload and sign a finding
2 View my findings
3 View findings shared with me
4 Share a finding with another researcher
5 Change password
6 Rotate my cryptographic keys
7 Logout
Choice: 1
Finding name : Clinical Validation of X-Ray Feature Detection
Finding text : The study cohort comprised 1,200 anonymized chest X-ray images
s consistent with early-stage pulmonary nodules with a sensitivity of 94.2% a
d pre-screening can significantly reduce radiologist workload in high-volume
1 deployment.
[OK] Finding uploaded and signed. ID: bcefa5b9-c24e-4584-b2a5-05c6443d97ba

Choice: 2
[0] Clinical Validation of X-Ray Feature Detection (ID: bcefa5b9-c24e-4584-b2a5-05c6443d97ba)
Select: 0

Content : The study cohort comprised 1,200 anonymized chest X-ray images. Our automated feature
consistent with early-stage pulmonary nodules with a sensitivity of 94.2% and a specificity of 91.5
pre-screening can significantly reduce radiologist workload in high-volume settings. Ongoing valida
deployment.
Signature : VALID ✓

```

Figure 2.3.1: Researcher-role clinical data upload and cryptographic integrity verification. Demonstrates the end-to-end lifecycle of a research finding, showing the automated signing at upload and the re-hashing and signature validation process during retrieval.

```

1 Upload patient dataset
2 View my datasets
3 View datasets shared with me
4 Share a dataset with another clinician
5 Erase a dataset
6 Change password
7 Rotate my cryptographic keys
8 Logout
Choice: 1
Patient ID : PT-2026-0213
Dataset content : Female, age 46, referred for oncology review following abnormal mammography.
irregular lesion in the upper outer quadrant of the left breast with no confirmed nodal involvem
nding. Dataset prepared for authorised multidisciplinary research review and comparative diagnos
Consent basis : Explicit consent obtained for secondary research processing
[OK] Dataset encrypted and stored. ID: 36fa2d25-0f5c-4ed6-9cd9-7ca593ef93fc

Choice: 2
[0] dataset_849a13148ed0
[1] dataset_4fb2497ea8f3
Select: 1

Patient ID : PT-2026-0213
Data : Female, age 46, referred for oncology review following abnormal
rregular lesion in the upper outer quadrant of the left breast with no confirmed
ding. Dataset prepared for authorised multidisciplinary research review and comp
Consent basis : Explicit consent obtained for secondary research processing
Signature : VALID ✓

```

Figure 2.3.2: Clinical content upload and verification. Demonstrates the complete integrity cycle, from authenticated patient dataset upload to the successful validation of content hashes and cryptographic signatures upon retrieval.

2.4 Key Management

The key management subsystem is implemented in `key_manager.py`. Key bundle generation creates Ed25519 and X25519 keypairs, which are encrypted at rest using a KEK derived from the user's password. The `rewrap_all_user_private_keys()` function allows password updates without requiring identity rotation, while `rotate_user_keys()` safely retires old key versions, maintaining access to historical material through explicit versioning.

```
6 Rotate my cryptographic keys
7 Logout
Choice: 6

--- Rotate My Cryptographic Keys ---
This will create a new active signing and encryption key version.
Previous key versions will remain stored for historical verification.
Type ROTATE to continue: ROTATE
[OK] Keys rotated successfully. New active version: 2
[!] Please log in again to continue with the new active keys.
```

Figure 2.4.1: User-initiated key rotation. Confirms the successful creation of new key versions, ensuring that previous cryptographic material remains stored for historical verification while updating the active signing/encryption version.

```
6 Change password
7 Rotate my cryptographic keys
8 Logout
Choice: 6

--- Change Password ---
Current password:
New password:
Confirm new password:
OK Password changed successfully. Please log in again.
```

Figure 2.4.2: Password change flow. User-initiated password rotation flow, showing the system-level confirmation upon successful update of authentication credentials.

2.5 Audit Logging

The `audit_log.py` module manages the system's integrity through an HMAC-chained audit log. Significant actions such as file uploads and user creation are recorded via `log_event()`, which chains each entry to the previous HMAC. The `verify_log_integrity()` function allows auditors to confirm the chain's validity, while `sanitise_audit_detail()` ensures sensitive operational data is stripped before persistence.

```

--- Auditor Menu ---
1 View audit log
2 Verify audit log integrity
3 Verify a signature
4 Change password
5 Logout
Choice: 1

```

Timestamp	User	Role	Action
2026-05-03T16:54:06Z	SYSTEM	admin	BOOTSTRAP_ADMIN_CREATED
2026-05-03T16:59:03Z	SYSTEM	admin	USER_REGISTERED
2026-05-03T17:02:18Z	lily_mathew	researcher	LOGIN_SUCCESS
2026-05-03T17:05:25Z	lily_mathew	researcher	UPLOAD_FINDING
2026-05-03T17:09:04Z	lily_mathew	researcher	LOGOUT
2026-05-03T17:09:21Z	admin	admin	LOGIN_SUCCESS
2026-05-03T17:24:16Z	admin	admin	LOGIN_SUCCESS
2026-05-03T17:24:18Z	admin	admin	LIST_USERS
2026-05-03T17:25:04Z	admin	admin	LOGOUT
2026-05-03T17:26:35Z	SYSTEM	admin	USER_REGISTERED
2026-05-03T17:27:24Z	jakethomas	clinician	LOGIN_SUCCESS
2026-05-03T17:28:55Z	jakethomas	clinician	UPLOAD_DATASET
2026-05-03T17:33:04Z	jakethomas	clinician	UPLOAD_DATASET
2026-05-03T17:34:17Z	jakethomas	clinician	VIEW_DATASET
2026-05-03T17:36:27Z	jakethomas	clinician	VIEW_DATASET
2026-05-03T17:40:53Z	lily_mathew	researcher	LOGIN_SUCCESS
2026-05-03T17:41:26Z	lily_mathew	researcher	VIEW_FINDING
2026-05-03T17:45:00Z	lily_mathew	researcher	KEY_ROTATED
2026-05-03T17:49:54Z	SYSTEM	admin	USER_REGISTERED
2026-05-03T17:50:23Z	claral	auditor	LOGIN_SUCCESS

Figure 2.5.1: Auditor view of the HMAC-chained audit log. Displays the immutable log record, demonstrating how user activity is captured and chronologically linked via HMAC-chaining to ensure tamper-evidence.

2.6 Admin Functions

Administrative capabilities are defined in `admin_menu()` within `main.py`. Administrators can create and manage user accounts, adjust account statuses, and trigger `export_backup()` to generate a timestamped archive of the platform state. A safeguard prevents administrators from disabling their own account, ensuring administrative access cannot be accidentally revoked.

```

--- Admin Menu ---
1 List users
2 Create new user
3 Disable user
4 Enable user
5 Export system backup
6 Change password
7 Logout
Choice: 1

```

Username	Role	Status	Name	Location	Pwd Changed
admin	admin	active	System Administrator	Internal	2026-05-03 17:54
lily_mathew	researcher	active	Lily Mathew	Manchester, UK	2026-05-03 17:59
jakethomas	clinician	active	Jake Thomas	Georgia, US	2026-05-03 18:58
claral	auditor	active	Clara Leon	Berlin, Germany	2026-05-03 18:49

Figure 2.6.1: Admin User List. Administrative view of the registered user base, including role, account status, and credential management metadata.

```
--- Admin Menu ---
1 List users
2 Create new user
3 Disable user
4 Enable user
5 Export system backup
6 Change password
7 Logout
Choice: 5
[OK] Backup created: C:\Users\MADHU\Downloads\crypto_system\backups\admin_20260503_190852.zip
```

Figure 2.6.2: Admin Menu Backup. Administrative system control interface showing backup export functionality for platform disaster recovery.

3 Testing and Legal Compliance

This section confirms that the Aurelia platform meets both functional security requirements and GDPR mandates through targeted testing.

3.1 Test Accounts

The platform includes pre-configured accounts to demonstrate role-based separation. All accounts maintain active status unless manually disabled during testing.

Username	Role	Password
admin	Administrator	Admin@2004
lily_mathew	Researcher	Waterlily@10
jakethomas	Clinician	Applepie*19
claral	Auditor	Mocha67\$
celinefred	Researcher	Tulips!1990

3.2 Functional Feature Tests

Functional testing confirms that core platform behaviours operate as designed under normal conditions.

- **Signature Verification:** Verification of untampered content returns a VALID ✓ status. Conversely, files with modified signatures trigger an INVALID signature alert.

```

3 Verify a signature
4 Change password
5 Logout
Choice: 3
[0] Clinical Validation of X-Ray Feature Detection by lily_mathew (researcher)
[1] 7278a52d by jakethomas (clinician)
[2] 36fa2d25 by jakethomas (clinician)
Select file: 2

File ID      : 36fa2d25-0f5c-4ed6-9cd9-7ca593ef93fc
Owner       : jakethomas
Role        : clinician
File Ref     : 36fa2d25
Hash        : 0a8be7345ce58e00...
Signature    : VALID ✓

```

Figure 3.2.1: Auditor signature verification. Auditor-role verification of clinical content showing a VALID signature for untampered data, confirming that the stored hash matches the signed file digest.

```

Choice: 3
[0] Genomic Sequence Variant Analysis: BRCA1/2 by celinefred (researcher)
[1] Clinical Validation of X-Ray Feature Detection by lily_mathew (researcher)
[2] 7278a52d by jakethomas (clinician)
Select file: 1

File ID      : bcefa5b9-c24e-4584-b2a5-05c6443d97ba
Owner       : lily_mathew
Role        : researcher
Finding     : Clinical Validation of X-Ray Feature Detection
Hash        : eb0e28f5f3c55e3a...
Signature    : INVALID ✗

```

Figure 3.2.2: Invalid signature detection. Verification of modified clinical content returns an INVALID signature result, demonstrating that integrity checks fail when the file hash no longer matches the stored signature.

- **Shared File Access:** Recipient researcher successfully views and verifies shared clinical finding.

<pre> [+] Welcome, Celine Frederick Role : researcher Location : London, UK --- Researcher Menu --- 1 Upload and sign a finding 2 View my findings 3 View findings shared with me 4 Share a finding with another researcher 5 Change password 6 Rotate my cryptographic keys 7 Logout Choice: 4 [0] Genomic Sequence Variant Analysis: BRCA1/2 Select finding to share: 0 Share with (username): lily_mathew [OK] Finding shared with lily_mathew. </pre>	<pre> [+] Welcome, Lily Mathew Role : researcher Location : Manchester, UK --- Researcher Menu --- 1 Upload and sign a finding 2 View my findings 3 View findings shared with me 4 Share a finding with another researcher 5 Change password 6 Rotate my cryptographic keys 7 Logout Choice: 3 [0] Genomic Sequence Variant Analysis: BRCA1/2 from celinefred Select: 0 Content : Sequencing analysis of the BRCA1 and BRCA2 genes was t (c.5266dupC) in 3.2% of the cohort, correlating strongly with ea uencing. The data suggests that targeted screening protocols shoul es. Signature : VALID ✓ </pre>
--	---

Figure 3.2.3: Recipient-role view of shared dataset with independent signature validation.

- **Password-Independent Decryption:** User changes password; existing files remain decryptable via rewrapped KEK.

```
[+] Welcome, Celine Frederick
Role : researcher
Location : London, UK

--- Researcher Menu ---
1 Upload and sign a finding
2 View my findings
3 View findings shared with me
4 Share a finding with another researcher
5 Change password
6 Rotate my cryptographic keys
7 Logout
Choice: 5

--- Change Password ---
Current password:
New password:
Confirm new password:
OK Password changed successfully. Please log in again.

1 Login
2 Register new account
3 Exit
Option: 1
Username : celinefred
Password :
[DEMO] Current MFA code for celinefred: 632211
Authenticator code : 632211
[+] Welcome, Celine Frederick

[DEMO] Current MFA code for celinefred: 632211
Authenticator code : 632211

[+] Welcome, Celine Frederick
Role : researcher
Location : London, UK

--- Researcher Menu ---
1 Upload and sign a finding
2 View my findings
3 View findings shared with me
4 Share a finding with another researcher
5 Change password
6 Rotate my cryptographic keys
7 Logout
Choice: 2
[0] Genomic Sequence Variant Analysis: BRCA1/2 (ID: d9
Select: 0

Content : Sequencing analysis of the BRCA1 and BRCA2
t (c.5266dupC) in 3.2% of the cohort, correlating strong
uencing. The data suggests that targeted screening proto
es.
Signature : VALID ✓
```

Figure 3.2.4: Post-password-change file access. celinefred decrypts previously uploaded dataset after password rotation, confirming `rewrap_all_user_private_keys()` success.

- **Audit Chain Integrity:** The `verify_log_integrity()` function confirms the HMAC-chained sequence of events, ensuring every entry is cryptographically bound to its predecessor.

```
--- Auditor Menu ---
1 View audit log
2 Verify audit log integrity
3 Verify a signature
4 Change password
5 Logout
Choice: 2
[OK] Log chain intact.
```

Figure 3.2.5: Auditor log chain. Independent verification of the HMAC-chained audit log demonstrating tamper-evident record keeping.

3.3 Access Control and RBAC Tests

These tests focus on boundary enforcement and defensive security, confirming that the platform denies unauthorized access.

- **Session Management:** The system enforces idle timeouts, requiring re-authentication after `SESSION_TIMEOUT_SECONDS`.


```

--- Researcher Menu ---
1 Upload and sign a finding
2 View my findings
3 View findings shared with me
4 Share a finding with another researcher
5 Change password
6 Rotate my cryptographic keys
7 Logout
Choice:
[!] Unrecognised option.
[!] Session expired. Please log in again.

1 Login
2 Register new account
3 Exit

```

Figure 3.3.1: Session expires after 900 seconds of inactivity as defined by `SESSION_TIMEOUT_SECONDS` in `config.py`, returning the user to the login prompt.

- **RBAC Boundaries:** Attempts by clinicians to share findings with researchers and vice versa are programmatically blocked, enforcing role-scoped sharing rules.

<pre> --- Researcher Menu --- 1 Upload and sign a finding 2 View my findings 3 View findings shared with me 4 Share a finding with another researcher 5 Change password 6 Rotate my cryptographic keys 7 Logout Choice: 4 [0] Clinical Validation of X-Ray Feature Detection Select finding to share: 0 Share with (username): jakethomas [!] Can only share with other researchers. </pre>	<pre> 1 Upload patient dataset 2 View my datasets 3 View datasets shared with me 4 Share a dataset with another clinician 5 Erase a dataset 6 Change password 7 Rotate my cryptographic keys 8 Logout Choice: 4 [0] dataset_849a13148ed0 [1] dataset_4fb2497ea8f3 Select dataset to share: 0 Share with (username): celinefred [!] Can only share with other clinicians. </pre>
---	---

Figure 3.3.2: Role-scoped sharing restriction. Clinician-to-researcher and researcher-to-clinician share attempts blocked with `ROLE_MISMATCH` error, enforcing brief-mandated scoping rules.

- **Account Lockout:** After three consecutive authentication failures, the system enforces a temporary lockout. The login flow displays the remaining seconds before access is restored.

<pre> 1 Login 2 Register new account 3 Exit Option: 1 Username : lily_mathew Password : [!] Login failed. Check your credentials. 1 Login 2 Register new account 3 Exit Option: 1 Username : lily_mathew Password : [!] Login failed. Check your credentials. </pre>	<pre> 1 Login 2 Register new account 3 Exit Option: 1 Username : lily_mathew Password : [!] Login failed. Check your credentials. 1 Login 2 Register new account 3 Exit Option: 1 Username : lily_mathew Password : [!] Account locked. Try again in 284 seconds. </pre>
---	---

Figure 3.3.3: Third consecutive failure triggers `LOGIN_BLOCKED` with a 284-second countdown, matching the `LOCKOUT_DURATION` constant and confirming the event is captured in the audit log.

- **Disabled Account:** A user account disabled by an administrator is immediately prevented from authenticating, confirming that account_status checks are enforced at login.

```

--- Admin Menu ---
1 List users
2 Create new user
3 Disable user
4 Enable user
5 Export system backup
6 Change password
7 Logout
Choice: 3
Username to disable: celinefred
[OK] User 'celinefred' has been disabled.

1 Login
2 Register new account
3 Exit
Option: 1
Username : celinefred
Password :
[!] Login failed. Check your credentials.

```

Figure 3.3.4: Disabled account authentication block. Administrator-disabled account test showing that a deactivated user is immediately prevented from logging in.

- **MFA Rejection:** Submission of an incorrect TOTP code is rejected independently of password verification, confirming that both factors are required.

```

1 Login
2 Register new account
3 Exit
Option: 1
Username : jakethomas
Password :
[DEMO] Current MFA code for jakethomas: 322513
Authenticator code : 322512
[!] Invalid authenticator code.

```

Figure 3.3.5: MFA rejection. Login test demonstrating that an incorrect TOTP code is rejected even when the primary password is valid, confirming two-factor enforcement.

- **Privilege Protection:** Administrative functions include safeguards that prevent system administrators from disabling their own accounts.

```

--- Admin Menu ---
1 List users
2 Create new user
3 Disable user
4 Enable user
5 Export system backup
6 Change password
7 Logout
Choice: 3
Username to disable: admin
[!] You cannot disable your own admin account.

```

Figure 3.3.6: Privilege protection. Administrative safeguard test showing that the system prevents administrators from disabling their own accounts.

- Right to Erasure: Clinician deletes a dataset via `delete_file()`, and the file is removed from both the clinician's dataset view and the auditor's verification list.

```

1 Upload patient dataset
2 View my datasets
3 View datasets shared with me
4 Share a dataset with another clinician
5 Erase a dataset
6 Change password
7 Rotate my cryptographic keys
8 Logout
Choice: 5
  [0] dataset_849a13148ed0
  [1] dataset_4fb2497ea8f3
Select to erase: 1
[OK] Dataset erased.

--- Auditor Menu ---
1 View audit log
2 Verify audit log integrity
3 Verify a signature
4 Change password
5 Logout
Choice: 3
  [0] Clinical Validation of X-Ray Feature Detection by lily_mathew (researcher)
  [1] Genomic Sequence Variant Analysis: BRCA1/2 by celinefred (researcher)
  [2] 7278a52d by jakethomas (clinician)
Select file:

```

Figure 3.3.7: Dataset erasure confirmation. `delete_file()` success for clinician jakethomas, with the dataset no longer visible in the clinician view or the clara's (auditor) verified file list.

3.4 Data Integrity Tests

Integrity testing proves that the system detects unauthorized modifications to persisted data.

- **Tamper-Evidence:** A manual modification to an entry in the `audit_log` table within `clinical_platform.db` triggers a chain verification failure. The verification tool accurately identifies the specific entry point where the HMAC chain was compromised. entry_id 4 was modified directly in the `audit_log` table to simulate attacker storage access.

```

[+] Welcome, Clara Leon
Role    : auditor
Location : Berlin, Germany

--- Auditor Menu ---
1 View audit log
2 Verify audit log integrity
3 Verify a signature
4 Change password
5 Logout
Choice: 2
[WARN] Chain broken at entries: [4]

```

Figure 3.4.1: Audit log tamper detection. Manual database modification test showing that the HMAC chain verification fails at the point where the audit log was altered.

- **Cryptographic Epoch Isolation:** Following key rotation, files wrapped under a previous X25519 keypair become inaccessible to the owner via the decryption path, as confirmed by the UNWRAP_FAIL and DECRYPT_FAIL entries in the audit log. However, Ed25519 signature verification remains unaffected, as the original public signing key is stored persistently in the files table and used directly by the auditor without any dependency on the current active keypair. This confirms that authenticity verification is fully decoupled from the decryption path.

```

--- Researcher Menu ---
1 Upload and sign a finding
2 View my findings
3 View findings shared with me
4 Share a finding with another researcher
5 Change password
6 Rotate my cryptographic keys
7 Logout
Choice: 2
  [0] Clinical Validation of X-Ray Feature Detection (ID
Select: 0
[!] Decryption failed or ciphertext was tampered with.

Choice: 3
  [0] Genomic Sequence Variant Analysis: BRCA1/2 by celinefred (researcher)
  [1] Clinical Validation of X-Ray Feature Detection by lily_mathew (researcher)
  [2] 7278a52d by jakethomas (clinician)
Select file: 1

File ID      : bcefa5b9-c24e-4584-b2a5-05c6443d97ba
Owner       : lily_mathew
Role        : researcher
Finding     : Clinical Validation of X-Ray Feature Detection
Hash        : eb0e28f5f3c55e3a...
Signature    : VALID ✓

```

Figure 3.4.2: Decryption fails with InvalidTag after key rotation because the wrapped file key targets the retired X25519 public key; the auditor's signature verification succeeds independently using the stored ed25519_public column, confirming the two paths are cryptographically decoupled.

3.5 Legal Compliance Mapping

This mapping confirms that all platform features tested above are aligned with the regulatory requirements outlined in Section 1.

Requirement	Article	Technical Control	Test Evidence	Analytical commentary
Encrypted storage	Art. 32	AES-256-GCM	Fig 3.2.1	AES-GCM protects confidentiality and integrity in one step, making it a strong fit for clinical data.
Access control	Art. 32	RBAC + MFA	Figs 3.3.1-3.3.7	RBAC plus MFA limits access to authenticated users in the correct role, reducing misuse risk.
DPIA support	Art. 35	Tamper-evident audit log	Fig 2.5.1	The audit log gives traceable evidence of system activity, which supports a DPIA.
Data minimisation	Art. 5(1)(c)	Sanitised audit log	Fig 2.5.1	Removing raw patient identifiers keeps only necessary operational data.

Lawful basis	Art. 9	consent_basis field	Fig 2.3.2	The consent_basis field records the legal basis directly in the system.
Right to erasure	Art. 17	delete_file()	Fig 3.3.7	delete_file() removes the file and sharing records, so withdrawal can be enforced technically.
Audit trail	Art. 5(2)	HMAC chain	Fig 3.4.1	HMAC chaining makes tampering detectable and strengthens accountability.

4 Reflection

Developing Aurelia showed that secure system design is not just about selecting strong primitives, but about ensuring they interoperate correctly across authentication, key management, access control, and audit. The main technical achievement was cryptographic epoch isolation: by persisting historical public signing keys independently of the active key state, signature verification remains valid after key rotation without depending on the decryption path. This highlighted the tension between key agility and data accessibility.

A genuine limitation of the current system is that administrative authority remains concentrated in a single privileged role. Sensitive operations, account control, backup export, and key lifecycle changes would require dual-approval or policy-enforced authorisation in a production deployment. The backup subsystem is also too coarse, supporting only full-system export rather than dataset-level or user-scoped recovery. They affect operational resilience and would present real risks under GDPR Article 5(2) accountability if the administrative account were compromised.

Prototype convenience can silently undermine security assumptions. Displaying the TOTP secret at enrolment and making key rotation user-initiated were both necessary for testing, but neither is acceptable in a production clinical system. A real deployment would provision TOTP via a secure out-of-band channel and enforce key rotation at defined cryptoperiods, consistent with NIST SP 800-57 recommendations on cryptographic key lifecycle management (Dworkin, 2007; OWASP Foundation, 2023). Recognising where a prototype diverges from production hardening is itself a security skill.

The design was shaped by the module's treatment of authenticated encryption modes, elliptic-curve key agreement, and HMAC-based integrity verification (Taramonli, 2026). The lecture material on why authenticated encryption must be treated as a single primitive, rather than composed from separate confidentiality and MAC layers, drove the selection of AES-GCM over AES-CBC with a separate HMAC. Similarly, the discussion of nonce-reuse vulnerabilities in ECDSA informed the move to Ed25519. AI tools were used only for wording and structural refinement; all technical decisions and final claims were verified against the implemented system (see Appendix A for full disclosure).

5 References

- Biryukov, A., Dinu, D. and Khovratovich, D. (2021) *Argon2: Memory-Hard Function for Password Hashing and Proof-of-Work Applications*. Internet Engineering Task Force (IETF). RFC 9106. Available at: <https://datatracker.ietf.org/doc/html/rfc9106> (Accessed: 27 April 2026).
- Dworkin, M.J. (2007) *SP 800-38D: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. National Institute of Standards and Technology (NIST). Available at: <https://csrc.nist.gov/publications/detail/sp/800-38d/final> (Accessed: 27 April 2026).
- European Parliament and Council of the European Union (2016) *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)*. *Official Journal of the European Union*, L 119, pp. 1–88.
- Information Commissioner's Office (ICO) (2024) *Guide to the UK GDPR: Article 9 – Processing of special categories of personal data*. Available at: <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/> (Accessed: 27 April 2026).
- Josefsson, S. and Liusvaara, I. (2017) *RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA)*. Internet Engineering Task Force (IETF). Available at: <https://datatracker.ietf.org/doc/html/rfc8032> (Accessed: 27 April 2026).
- Langley, A., Hamburg, M. and Turner, S. (2016) *RFC 7748: Elliptic Curves for Diffie-Hellman Key Agreement (X25519 and X448)*. Internet Engineering Task Force (IETF). Available at: <https://datatracker.ietf.org/doc/html/rfc7748> (Accessed: 27 April 2026).
- Microsoft STRIDE Team (2009) *The STRIDE Threat Model*. Microsoft Security Development Lifecycle. Available at: [https://docs.microsoft.com/en-us/previous-versions/commerce-server/ee823878\(v=cs.20\)](https://docs.microsoft.com/en-us/previous-versions/commerce-server/ee823878(v=cs.20)) (Accessed: 27 April 2026).
- OWASP Foundation (2023) *Password Storage Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (Accessed: 27 April 2026).
- OWASP Foundation (2022) *Cryptographic Storage Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html (Accessed: 27 April 2026).
- PyCA (2026) *Cryptography: Python Cryptographic Recipes and Primitives - Documentation*. Available at: <https://cryptography.io/en/latest/> (Accessed: 27 April 2026).
- PyOTP (2026) *PyOTP - The Python One-Time Password Library*. Available at: <https://github.com/pyauth/pyotp> (Accessed: 27 April 2026).

Shostack, A. (2014) *Threat Modeling: Designing for Security*. Indianapolis: Wiley.

Taramonli, S. (2026) *Applied Cryptography (WM9PC-15): Design and Implementation Lectures*. WMG, University of Warwick.

6 Appendices

Appendix A

This assessment was completed under the AI-Collaboration category, which permits AI to assist with specific tasks such as drafting, refining, and evaluating work, provided that any generated content is critically reviewed and modified before submission. AI was used in a limited support role during report development, primarily to help improve structural clarity, refine academic phrasing, generic template-style or scaffold code (which was then reviewed, adapted, and integrated manually) and identify possible production-oriented reflection points.

AI was not used to replace the core design, implementation, or testing decisions in the coursework. The cryptographic architecture, Python implementation, screenshots, test evidence, and final technical claims were determined through the student's own development work and then checked against the assignment brief and rubric requirements before inclusion in the report.

The main AI-supported activities are summarised below:

AI-assisted task	Prompt theme	How the output was used
Report structure	Suggest a clearer report flow for design, implementation, testing, and reflection sections.	Used to improve section ordering and alignment with the brief; the final structure was selected manually.
Legal and ethical wording	Help phrase GDPR, consent, and compliance points in a precise academic style.	Used to draft wording, then reviewed and adapted to match the implemented system and evidence.
Reflection support	Generate production-oriented improvement themes for a prototype reflection.	Used to identify possible future improvements, then rewritten into the final report style.
Disclosure drafting	Draft a concise AI-use disclosure suitable for an academic appendix.	Used to structure this appendix, then edited to accurately describe the actual extent of AI use.
General language refinement	Improve clarity and concision in technical explanations.	Used for wording support only; all technical meaning was checked and revised manually.

All AI-assisted outputs were reviewed critically before use. Content was retained only where it accurately matched the implemented system behaviour, screenshots, testing results, and legal/compliance discussion already evidenced elsewhere in the report. Any wording that

overstated functionality, implied unsupported evidence, or reduced technical precision was revised or discarded before final submission.

Appendix B

This appendix presents the complete, modularised Python implementation of the Aurelia Clinical Research Platform, structured logically to reflect the core security mechanisms, cryptographic protocols, and access control boundaries discussed in the main technical report.

B.1 Authentication and Credential Security

Implementation of credential hashing, TOTP validation, and account lockout mechanisms.

B.1.1 Password Validation and Registration

```
# auth.py
# Registration, password verification, TOTP, account lockout, and password change.
import time
import re
import pyotp
from argon2 import PasswordHasher
from argon2.exceptions import VerifyMismatchError, VerificationError,
InvalidHashError

from storage import (
    create_user,
    get_user,
    update_password_hash,
    update_failed_attempts,
    reset_failed_attempts,
)
from key_manager import create_initial_keys, rewrap_all_user_private_keys
from config import (
    ARGON2_TIME_COST,
    ARGON2_MEMORY_COST,
    ARGON2_PARALLELISM,
    MAX_FAILED_ATTEMPTS,
    LOCKOUT_DURATION,
    TOTP_ISSUER,
)

ph = PasswordHasher(
    time_cost=ARGON2_TIME_COST,
    memory_cost=ARGON2_MEMORY_COST,
    parallelism=ARGON2_PARALLELISM,
)

VALID_ROLES = {"researcher", "clinician", "auditor", "admin"}

def validate_password_strength(password: str) -> None:
    if len(password) < 8:
        raise ValueError("Password must be at least 8 characters long.")
```



```

    if not re.search(r"[A-Z]", password):
        raise ValueError("Password must contain at least one uppercase letter.")
    if not re.search(r"[a-z]", password):
        raise ValueError("Password must contain at least one lowercase letter.")
    if not re.search(r"\d", password):
        raise ValueError("Password must contain at least one digit.")
    if not re.search(r"[!@#$%^&*()_\-+=\[\]\{\};':\"\\|,.<>/?]", password):
        raise ValueError("Password must contain at least one special character.")

def register_user(
    username: str,
    password: str,
    role: str,
    first_name: str = "",
    last_name: str = "",
    location: str = "",
) -> str:
    if role not in VALID_ROLES:
        raise ValueError(f"Invalid role: {role}")

    if get_user(username):
        raise ValueError(f"Username '{username}' already exists.")

    validate_password_strength(password)

    password_hash = ph.hash(password)
    totp_secret = pyotp.random_base32()
    now = time.time()

    create_user(
        username=username,
        password_hash=password_hash,
        role=role,
        totp_secret=totp_secret,
        created_at=now,
        first_name=first_name,
        last_name=last_name,
        location=location,
    )

    create_initial_keys(username, password)
    return totp_secret

```

B.1.2 Multi-Factor Authentication and Session Lockout

```

def get_totp_uri(username: str, secret: str) -> str:
    return pyotp.TOTP(secret).provisioning_uri(
        name=username,
        issuer_name=TOTP_ISSUER,
    )

def verify_totp(username: str, token: str) -> bool:

```

```

    user = get_user(username)
    if not user or user.get("account_status") != "active":
        return False
    return pyotp.TOTP(user["totp_secret"]).verify(token, valid_window=1)

def authenticate(username: str, password: str):
    user = get_user(username)
    if not user:
        return None

    if user.get("account_status", "active") != "active":
        return None

    if user["locked_until"] > time.time():
        remaining = int(user["locked_until"] - time.time())
        return f"locked:{remaining}"

    try:
        ph.verify(user["password_hash"], password)
        reset_failed_attempts(username)
        return user
    except (VerifyMismatchError, VerificationError, InvalidHashError):
        attempts = user["failed_attempts"] + 1
        locked_until = 0

        if attempts >= MAX_FAILED_ATTEMPTS:
            locked_until = time.time() + LOCKOUT_DURATION

        update_failed_attempts(username, attempts, locked_until)
        return None

def change_password(username: str, old_password: str, new_password: str):
    user = get_user(username)
    if not user:
        raise ValueError("User not found.")

    try:
        ph.verify(user["password_hash"], old_password)
    except (VerifyMismatchError, VerificationError, InvalidHashError):
        raise ValueError("Current password is incorrect.")

    validate_password_strength(new_password)

    rewrap_all_user_private_keys(username, old_password, new_password)
    update_password_hash(username, ph.hash(new_password))

```

B.2 Role-Based Access Control

Role-specific operational loops enforcing access control boundaries for clinical and audit functions.

B.2.1 Researcher Operations

```
def researcher_menu(username: str, keys: dict, session_start: float, password:
str) -> None:
    role = "researcher"

    while True:
        if not check_session(session_start):
            print(colour("[!] Session expired. Please log in again.", RED))
            log_event(username, role, "SESSION_TIMEOUT", "")
            break

        print("\n--- Researcher Menu ---")
        print("1 Upload and sign a finding")
        print("2 View my findings")
        print("3 View findings shared with me")
        print("4 Share a finding with another researcher")
        print("5 Change password")
        print("6 Rotate my cryptographic keys")
        print("7 Logout")
        choice = input("Choice: ").strip()

        if choice == "1":
            filename = input("Finding name : ").strip()
            content = input("Finding text : ").encode("utf-8")
            content_hash = hashlib.sha256(content).hexdigest()

            signature = sign_data(keys["ed_private"], content_hash.encode("utf-
8"))

            blob = encrypt_file(content, keys["x_public"])

            file_id = str(uuid.uuid4())
            store_file(
                file_id=file_id,
                owner=username,
                role=role,
                filename=filename,
                ciphertext=blob["ciphertext"],
                nonce=blob["nonce"],
                signature=signature,
                content_hash=content_hash,
                ed25519_public=serialise_public_key(keys["ed_public"]),
                created_at=time.time(),
            )

            store_share(
                share_id=str(uuid.uuid4()),
                file_id=file_id,
                shared_by=username,
                shared_with=username,
                wrapped_file_key=blob["wrapped_file_key"],
                ephemeral_public=blob["ephemeral_public"],
```

```

        created_at=time.time(),
    )

    log_event(username, role, "UPLOAD_FINDING", f"file_id={file_id}")
    print(colour(f"[OK] Finding uploaded and signed. ID: {file_id}",
GREEN))

elif choice == "2":
    shares = get_shares_for_user(username)
    my_shares = [
        s for s in shares
        if s["shared_by"] == username and s["shared_with"] == username
    ]

    if not my_shares:
        print("No findings on record.")
        continue

    for i, s in enumerate(my_shares):
        print(f"  [{i}] {s['filename']} (ID: {s['file_id']})")

    try:
        idx = int(input("Select: "))
        if not (0 <= idx < len(my_shares)):
            print(colour("[!] Invalid selection.", RED))
            continue
    except ValueError:
        print(colour("[!] Enter a number.", RED))
        continue

    share = my_shares[idx]
    file_rec = get_file(share["file_id"])

    if not file_rec:
        print(colour("[!] File record missing.", RED))
        log_event(username, role, "VIEW_FAIL",
f"file_missing={share['file_id']}")
        continue

    try:
        plaintext = decrypt_file(
            {
                "ciphertext": file_rec["ciphertext"],
                "nonce": file_rec["nonce"],
                "wrapped_file_key": json.loads(share["wrapped_file_key"]),
                "ephemeral_public": share["ephemeral_public"],
            },
            keys["x_private"],
        )
    except (InvalidTag, ValueError, KeyError, json.JSONDecodeError) as e:

```

```

        print(colour("[!] Decryption failed or ciphertext was tampered
with.", RED))
        log_event(username, role, "DECRYPT_FAIL",
f"file_id={share['file_id']} error={type(e).__name__}")
        continue

        pub_key = deserialise_public_key(file_rec["ed25519_public"])
        computed_hash = hashlib.sha256(plaintext).hexdigest()
        sig_valid = (
            computed_hash == file_rec["content_hash"]
            and verify_signature(pub_key,
file_rec["content_hash"].encode("utf-8"), file_rec["signature"])
        )

        print(f"\n Content : {plaintext.decode('utf-8')}")
        status_text = colour("VALID ✓", GREEN) if sig_valid else
colour("INVALID ✗", RED)
        print(f" Signature : {status_text}")

        log_event(username, role, "VIEW_FINDING",
f"file_id={share['file_id']}")

    elif choice == "3":
        shares = get_shares_for_user(username)
        received = [
            s for s in shares
            if s["shared_with"] == username and s["shared_by"] != username
        ]

        if not received:
            print("No findings shared with you.")
            continue

        for i, s in enumerate(received):
            print(f" [{i}] {s['filename']} from {s['shared_by']}")

        try:
            idx = int(input("Select: "))
            if not (0 <= idx < len(received)):
                print(colour("[!] Invalid selection.", RED))
                continue
        except ValueError:
            print(colour("[!] Enter a number.", RED))
            continue

        share = received[idx]
        file_rec = get_file(share["file_id"])

        if not file_rec:
            print(colour("[!] File record missing.", RED))
            continue

```

```

        try:
            wrapped = json.loads(share["wrapped_file_key"]) if
isinstance(share["wrapped_file_key"], str) else share["wrapped_file_key"]
            plaintext = decrypt_file(
                {
                    "ciphertext": file_rec["ciphertext"],
                    "nonce": file_rec["nonce"],
                    "wrapped_file_key": wrapped,
                    "ephemeral_public": share["ephemeral_public"],
                },
                keys["x_private"],
            )
        except (InvalidTag, ValueError, KeyError, json.JSONDecodeError) as e:
            print(colour("[!] Decryption failed or ciphertext was tampered
with.", RED))
            log_event(username, role, "DECRYPT_FAIL",
f"file_id={share['file_id']} error={type(e).__name__}")
            continue

        pub_key = deserialise_public_key(file_rec["ed25519_public"])
        computed_hash = hashlib.sha256(plaintext).hexdigest()
        sig_valid = (
            computed_hash == file_rec["content_hash"]
            and verify_signature(pub_key,
file_rec["content_hash"].encode("utf-8"), file_rec["signature"])
        )

        print(f"\n Content : {plaintext.decode('utf-8')}")
        status_text = colour("VALID ✓", GREEN) if sig_valid else
colour("INVALID X", RED)
        print(f" Signature : {status_text}")

        log_event(
            username,
            role,
            "VIEW_SHARED_FINDING",
            f"file_id={share['file_id']}",
        )

    elif choice == "4":
        shares = get_shares_for_user(username)
        my_shares = [
            s for s in shares
            if s["shared_by"] == username and s["shared_with"] == username
        ]

        if not my_shares:
            print("No findings to share.")
            continue

```

```

for i, s in enumerate(my_shares):
    print(f" [{i}] {s['filename']}")

try:
    idx = int(input("Select finding to share: "))
    if not (0 <= idx < len(my_shares)):
        print(colour("[!] Invalid selection.", RED))
        continue
except ValueError:
    print(colour("[!] Enter a number.", RED))
    continue

recipient = input("Share with (username): ").strip()
recipient_user = get_user(recipient)

if not recipient_user:
    print(colour("[!] User not found.", RED))
    continue

if recipient_user["role"] != "researcher":
    print(colour("[!] Can only share with other researchers.", RED))
    continue

recipient_keys = get_user_active_public_keys(recipient)
if not recipient_keys:
    print(colour("[!] Recipient has no active keys on record.", RED))
    continue

share = my_shares[idx]

if share["shared_with"] != username:
    print(colour("[!] You are not authorised to re-share this item.",
RED))
    log_event(username, role, "RESHARE_DENIED",
f"file_id={share['file_id']}")
    continue

try:
    if isinstance(share["wrapped_file_key"], str):
        share = dict(share)
        share["wrapped_file_key"] =
json.loads(share["wrapped_file_key"])
        file_key = unwrap_file_key(share, keys["x_private"])
    except (InvalidTag, ValueError, KeyError, json.JSONDecodeError) as e:
        print(colour("[!] Could not unwrap file key.", RED))
        log_event(username, role, "UNWRAP_FAIL",
f"file_id={share['file_id']} error={type(e).__name__}")
        continue

    recipient_x_public =
deserialise_public_key(recipient_keys["x25519_public"])

```

```

        new_wrap = wrap_file_for_recipient(file_key, recipient_x_public)

        store_share(
            share_id=str(uuid.uuid4()),
            file_id=share["file_id"],
            shared_by=username,
            shared_with=recipient,
            wrapped_file_key=new_wrap["wrapped_file_key"],
            ephemeral_public=new_wrap["ephemeral_public"],
            created_at=time.time(),
        )

        log_event(username, role, "SHARE_FINDING",
f"file_id={share['file_id']} with={recipient}")
        print(colour(f"[OK] Finding shared with {recipient}.", GREEN))

    elif choice == "5":
        changed = password_change_flow(username, role)
        if changed:
            break

    elif choice == "6":
        rotated = self_key_rotation_flow(username, role, password)
        if rotated:
            break
    elif choice == "7":
        log_event(username, role, "LOGOUT", "")
        print("Logged out.")
        break

    else:
        print(colour("[!] Unrecognised option.", RED))

```

B.2.2 Clinician Operations

```

def clinician_menu(username: str, keys: dict, session_start: float, password: str)
-> None:
    role = "clinician"

    while True:
        if not check_session(session_start):
            print(colour("[!] Session expired. Please log in again.", RED))
            log_event(username, role, "SESSION_TIMEOUT", "")
            break

        print("1 Upload patient dataset")
        print("2 View my datasets")
        print("3 View datasets shared with me")
        print("4 Share a dataset with another clinician")
        print("5 Erase a dataset")
        print("6 Change password")
        print("7 Rotate my cryptographic keys")

```



```

print("8 Logout")
choice = input("Choice: ").strip()

if choice == "1":
    patient_id = input("Patient ID      : ").strip()
    data_content = input("Dataset content : ").strip()
    consent_basis = input("Consent basis   : ").strip()

    payload = json.dumps({
        "patient_id": patient_id,
        "data": data_content,
        "consent_basis": consent_basis
    }).encode("utf-8")

    content_hash = hashlib.sha256(payload).hexdigest()
    signature = sign_data(keys["ed_private"], content_hash.encode("utf-
8"))

    blob = encrypt_file(payload, keys["x_public"])
    file_id = str(uuid.uuid4())
    patient_ref = hashlib.sha256(patient_id.encode("utf-
8")).hexdigest()[:12]

    store_file(
        file_id=file_id,
        owner=username,
        role=role,
        filename=f"dataset_{patient_ref}",
        ciphertext=blob["ciphertext"],
        nonce=blob["nonce"],
        signature=signature,
        content_hash=content_hash,
        ed25519_public=serialise_public_key(keys["ed_public"]),
        created_at=time.time()
    )

    store_share(
        share_id=str(uuid.uuid4()),
        file_id=file_id,
        shared_by=username,
        shared_with=username,
        wrapped_file_key=blob["wrapped_file_key"],
        ephemeral_public=blob["ephemeral_public"],
        created_at=time.time()
    )

    patient_ref = hashlib.sha256(patient_id.encode("utf-
8")).hexdigest()[:12]
    log_event(
        username,
        role,
        "UPLOAD_DATASET",

```

```

        f"file_id={file_id} subject_ref={patient_ref}"
    )

    print(colour(f"[OK] Dataset encrypted and stored. ID: {file_id}",
GREEN))

    elif choice == "2":
        shares = get_shares_for_user(username)
        my_shares = [
            s for s in shares
            if s["shared_by"] == username and s["shared_with"] == username
        ]

        if not my_shares:
            print("No datasets on record.")
            continue

        for i, s in enumerate(my_shares):
            print(f"  [{i}] {s['filename']}")

        try:
            idx = int(input("Select: "))
            if not (0 <= idx < len(my_shares)):
                print(colour("[!] Invalid selection.", RED))
                continue
        except ValueError:
            print(colour("[!] Enter a number.", RED))
            continue

        share = my_shares[idx]
        file_rec = get_file(share["file_id"])

        if not file_rec:
            print(colour("[!] File record missing.", RED))
            continue

        try:
            wrapped = json.loads(share["wrapped_file_key"]) if
isinstance(share["wrapped_file_key"], str) else share["wrapped_file_key"]
            plaintext = decrypt_file(
                {
                    "ciphertext": file_rec["ciphertext"],
                    "nonce": file_rec["nonce"],
                    "wrapped_file_key": wrapped,
                    "ephemeral_public": share["ephemeral_public"]
                },
                keys["x_private"]
            )
        except (InvalidTag, ValueError, KeyError, json.JSONDecodeError) as e:
            print(colour("[!] Decryption failed or ciphertext was tampered
with.", RED))

```

```

        log_event(username, role, "DECRYPT_FAIL",
f"file_id={share['file_id']} error={type(e).__name__}")
        continue

        computed_hash = hashlib.sha256(plaintext).hexdigest()
        pub_key = deserialise_public_key(file_rec["ed25519_public"])
        sig_valid = (
            computed_hash == file_rec["content_hash"]
            and verify_signature(pub_key,
file_rec["content_hash"].encode("utf-8"), file_rec["signature"])
        )

        parsed = json.loads(plaintext)
        print(f"\n Patient ID      : {parsed['patient_id']}")
        print(f"  Data              : {parsed['data']}")
        print(f"  Consent basis : {parsed['consent_basis']}")
        status_text = colour("VALID ✓", GREEN) if sig_valid else
colour("INVALID X", RED)
        print(f"  Signature        : {status_text}")

        log_event(username, role, "VIEW_DATASET",
f"file_id={share['file_id']}")

    elif choice == "3":
        shares = get_shares_for_user(username)
        received = [
            s for s in shares
            if s["shared_with"] == username and s["shared_by"] != username
        ]

        if not received:
            print("No datasets shared with you.")
            continue

        for i, s in enumerate(received):
            print(f"  [{i}] {s['filename']} from {s['shared_by']}")

        try:
            idx = int(input("Select: "))
            if not (0 <= idx < len(received)):
                print(colour("[!] Invalid selection.", RED))
                continue
        except ValueError:
            print(colour("[!] Enter a number.", RED))
            continue

        share = received[idx]
        file_rec = get_file(share["file_id"])

        if not file_rec:
            print(colour("[!] File record missing.", RED))

```

```

        continue

    try:
        wrapped = json.loads(share["wrapped_file_key"]) if
isinstance(share["wrapped_file_key"], str) else share["wrapped_file_key"]
        plaintext = decrypt_file(
            {
                "ciphertext": file_rec["ciphertext"],
                "nonce": file_rec["nonce"],
                "wrapped_file_key": wrapped,
                "ephemeral_public": share["ephemeral_public"]
            },
            keys["x_private"]
        )
    except (InvalidTag, ValueError, KeyError, json.JSONDecodeError) as e:
        print(colour("[!] Decryption failed or ciphertext was tampered
with.", RED))
        log_event(username, role, "DECRYPT_FAIL",
f"file_id={share['file_id']} error={type(e).__name__}")
        continue

    computed_hash = hashlib.sha256(plaintext).hexdigest()
    pub_key = deserialise_public_key(file_rec["ed25519_public"])
    sig_valid = (
        computed_hash == file_rec["content_hash"]
        and verify_signature(pub_key,
file_rec["content_hash"].encode("utf-8"), file_rec["signature"])
    )

    parsed = json.loads(plaintext)
    print(f"\n Patient ID      : {parsed['patient_id']}")
    print(f"  Data              : {parsed['data']}")
    print(f"  Consent basis : {parsed['consent_basis']}")
    status_text = colour("VALID ✓", GREEN) if sig_valid else
colour("INVALID X", RED)
    print(f"  Signature          : {status_text}")

    log_event(
        username,
        role,
        "VIEW_SHARED_DATASET",
        f"file_id={share['file_id']}"
    )

elif choice == "4":
    shares = get_shares_for_user(username)
    my_shares = [
        s for s in shares
        if s["shared_by"] == username and s["shared_with"] == username
    ]

```

```

if not my_shares:
    print("No datasets to share.")
    continue

for i, s in enumerate(my_shares):
    print(f"  [{i}] {s['filename']}")

try:
    idx = int(input("Select dataset to share: "))
    if not (0 <= idx < len(my_shares)):
        print(colour("[!] Invalid selection.", RED))
        continue
except ValueError:
    print(colour("[!] Enter a number.", RED))
    continue

recipient = input("Share with (username): ").strip()
recipient_user = get_user(recipient)

if not recipient_user:
    print(colour("[!] User not found.", RED))
    continue

if recipient_user["role"] != "clinician":
    print(colour("[!] Can only share with other clinicians.", RED))
    continue

recipient_keys = get_user_active_public_keys(recipient)
if not recipient_keys:
    print(colour("[!] Recipient has no active keys.", RED))
    continue

share = my_shares[idx]

if share["shared_with"] != username:
    print(colour("[!] You are not authorised to re-share this item.",
RED))
    log_event(username, role, "RESHARE_DENIED",
f"file_id={share['file_id']}")
    continue

try:
    file_key = unwrap_file_key(share, keys["x_private"])
except (InvalidTag, ValueError, KeyError, json.JSONDecodeError) as e:
    print(colour("[!] Could not unwrap file key.", RED))
    log_event(username, role, "UNWRAP_FAIL",
f"file_id={share['file_id']} error={type(e).__name__}")
    continue

recipient_x_public =
deserialise_public_key(recipient_keys["x25519_public"])

```

```

new_wrap = wrap_file_for_recipient(file_key, recipient_x_public)

store_share(
    share_id=str(uuid.uuid4()),
    file_id=share["file_id"],
    shared_by=username,
    shared_with=recipient,
    wrapped_file_key=new_wrap["wrapped_file_key"],
    ephemeral_public=new_wrap["ephemeral_public"],
    created_at=time.time()
)

log_event(username, role, "SHARE_DATASET",
f"file_id={share['file_id']} with={recipient}")
print(colour(f"[OK] Dataset shared with {recipient}.", GREEN))

elif choice == "5":
    shares = get_shares_for_user(username)
    my_shares = [
        s for s in shares
        if s["shared_by"] == username and s["shared_with"] == username
    ]

    if not my_shares:
        print("No datasets to erase.")
        continue

    for i, s in enumerate(my_shares):
        print(f"  [{i}] {s['filename']}")

    try:
        idx = int(input("Select to erase: "))
        if not (0 <= idx < len(my_shares)):
            print(colour("[!] Invalid selection.", RED))
            continue
    except ValueError:
        print(colour("[!] Enter a number.", RED))
        continue

    file_id = my_shares[idx]["file_id"]
    delete_file(file_id)
    log_event(username, role, "ERASE_DATASET", f"file_id={file_id}")
    print(colour("[OK] Dataset erased.", GREEN))

elif choice == "6":
    changed = password_change_flow(username, role)
    if changed:
        break

elif choice == "7":
    rotated = self_key_rotation_flow(username, role, password)
    if rotated:

```

```

        break
    elif choice == "8":
        log_event(username, role, "LOGOUT", "")
        print("Logged out.")
        break

    else:
        print(colour("[!] Unrecognised option.", RED))

```

B.2.3 Auditor Operations

```

def auditor_menu(username: str, keys: dict, session_start: float) -> None:
    role = "auditor"

    SHOW_DETAIL_FOR = {
        "LOGIN_BLOCKED",
        "MFA_FAIL",
        "KEY_LOAD_FAIL",
        "DECRYPT_FAIL",
        "UNWRAP_FAIL",
        "RESHARE_DENIED",
        "VERIFY_LOG_INTEGRITY",
        "VERIFY_SIGNATURE",
        "ERASE_DATASET",
    }

    def get_visible_detail(entry: dict) -> str:
        action = entry.get("action", "")
        detail = entry.get("detail", "")

        if action not in SHOW_DETAIL_FOR:
            return ""

        if action == "VERIFY_SIGNATURE":
            return detail

        if action == "VERIFY_LOG_INTEGRITY":
            return detail

        if action == "LOGIN_BLOCKED":
            return detail

        if action in {"DECRYPT_FAIL", "UNWRAP_FAIL", "KEY_LOAD_FAIL"}:
            return detail

        if action == "RESHARE_DENIED":
            return "Unauthorised re-share attempt"

        if action == "ERASE_DATASET":
            return detail

        if action == "MFA_FAIL":

```

```

        return "Invalid MFA code"

    return ""

while True:
    if not check_session(session_start):
        print(colour("[!] Session expired.", RED))
        log_event(username, role, "SESSION_TIMEOUT", "")
        break

    print("\n--- Auditor Menu ---")
    print("1 View audit log")
    print("2 Verify audit log integrity")
    print("3 Verify a signature")
    print("4 Change password")
    print("5 Logout")
    choice = input("Choice: ").strip()

    if choice == "1":
        entries = read_log()

        if not entries:
            print("Audit log is empty.")
        else:
            visible_rows = []
            for e in entries:
                visible_rows.append({
                    "timestamp": e.get("timestamp", ""),
                    "user": e.get("user", ""),
                    "role": e.get("role", ""),
                    "action": e.get("action", ""),
                    "detail": get_visible_detail(e),
                })

            has_detail = any(row["detail"] for row in visible_rows)

            if has_detail:
                print(f"\n{'Timestamp':<22} {'User':<18} {'Role':<12} {'Action':<35} Detail")
                print("-" * 110)
                for row in visible_rows:
                    print(
                        f"{'Timestamp':<20} "
                        f"{'User':<18} "
                        f"{'Role':<12} "
                        f"{'Action':<35} "
                        f"{'Detail':<35}"
                    )
            else:
                print(f"\n{'Timestamp':<22} {'User':<18} {'Role':<12} {'Action':<35}")

```



```

        print("-" * 95)
        for row in visible_rows:
            print(
                f" {row['timestamp']:<20} "
                f"{row['user']:<18} "
                f"{row['role']:<12} "
                f"{row['action']:<35}"
            )

        log_event(username, role, "VIEW_AUDIT_LOG", "")

    elif choice == "2":
        valid, tampered_at = verify_log_integrity()
        if valid:
            print(colour("[OK] Log chain intact.", GREEN))
        else:
            print(colour(f"[WARN] Chain broken at entries: {tampered_at}",
RED))
            log_event(username, role, "VERIFY_LOG_INTEGRITY", f"valid={valid}")

    elif choice == "3":
        all_files = get_files_for_role("researcher") +
get_files_for_role("clinician")

        if not all_files:
            print("No signed files on record.")
            continue

        for i, f in enumerate(all_files):
            label = f["filename"] if f["role"] == "researcher" else
f["file_id"][:8]
            print(f" [{i}] {label} by {f['owner']} ({f['role']})")

        try:
            idx = int(input("Select file: "))
            if not (0 <= idx < len(all_files)):
                print(colour("[!] Invalid selection.", RED))
                continue
        except ValueError:
            print(colour("[!] Enter a number.", RED))
            continue

        file_rec = all_files[idx]
        pub_key = deserialise_public_key(file_rec["ed25519_public"])
        sig_valid = verify_signature(
            pub_key,
            file_rec["content_hash"].encode("utf-8"),
            file_rec["signature"]
        )

        print()

```

```

        print(f"File ID      : {file_rec['file_id']}")
        print(f"Owner       : {file_rec['owner']}")
        print(f"Role        : {file_rec['role']}")

        if file_rec["role"] == "researcher":
            print(f"Finding    : {file_rec['filename']}")
        else:
            print(f"File Ref   : {file_rec['file_id'][:8]}")

        hash_short = file_rec["content_hash"][:16] + "..."
        print(f"Hash       : {hash_short}")
        status_text = colour("VALID ✓", GREEN) if sig_valid else
colour("INVALID ✗", RED)
        print(f"Signature  : {status_text}")

        log_event(
            username,
            role,
            "VERIFY_SIGNATURE",
            f"file_id={file_rec['file_id']} valid={sig_valid}"
        )

    elif choice == "4":
        changed = password_change_flow(username, role)
        if changed:
            break
    elif choice == "5":
        log_event(username, role, "LOGOUT", "")
        print("Logged out.")
        break

    else:
        print(colour("[!] Unrecognised option.", RED))

```

B.3 File Encryption and Signing

Core cryptographic primitives for hybrid file encryption, digital signatures, and key agreement.

B.3.1 Symmetric Encryption and Signatures

```

# crypto_core.py
# Core cryptographic operations.
# Uses AES-256-GCM, Ed25519, X25519, and HKDF-SHA256.

import os
import base64
import json
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PrivateKey
from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.kdf.hkdf import HKDF

```

```

from cryptography.hazmat.backends import default_backend
from cryptography.exceptions import InvalidSignature
# -----
# AES-256-GCM helpers
# -----
def encrypt_data(key: bytes, plaintext: bytes) -> dict:
    """Encrypt plaintext with AES-GCM and return a JSON-safe blob."""
    nonce = os.urandom(12)
    ciphertext = AESGCM(key).encrypt(nonce, plaintext, None)
    return {
        "nonce": base64.b64encode(nonce).decode(),
        "ciphertext": base64.b64encode(ciphertext).decode()
    }

def decrypt_data(key: bytes, blob: dict) -> bytes:
    """Decrypt an AES-GCM blob. Raises on tamper or wrong key."""
    nonce = base64.b64decode(blob["nonce"])
    ciphertext = base64.b64decode(blob["ciphertext"])
    return AESGCM(key).decrypt(nonce, ciphertext, None)
# -----
# Ed25519 signing
# -----
def generate_ed25519_keypair():
    """Generate an Ed25519 keypair for signatures."""
    private_key = Ed25519PrivateKey.generate()
    return private_key, private_key.public_key()

def sign_data(private_key, data: bytes) -> str:
    """Sign data and return a base64 signature."""
    return base64.b64encode(private_key.sign(data)).decode()

def verify_signature(public_key, data: bytes, signature_b64: str) -> bool:
    """Verify an Ed25519 signature. Return True if valid."""
    try:
        public_key.verify(base64.b64decode(signature_b64), data)
        return True
    except (InvalidSignature, Exception):
        return False

```

B.3.2 Key Agreement and Wrapping

```

# -----
# X25519 key agreement
# -----
def generate_x25519_keypair():
    """Generate an X25519 keypair for key agreement."""
    private_key = X25519PrivateKey.generate()
    return private_key, private_key.public_key()

def derive_shared_secret(private_key, peer_public_key) -> bytes:
    """Run X25519 Diffie-Hellman and return the shared secret."""
    return private_key.exchange(peer_public_key)

```

```

def derive_file_wrapping_key(shared_secret: bytes, salt: bytes) -> bytes:
    """Derive a 256-bit wrapping key from the shared secret using HKDF-SHA256."""
    return HKDF(
        algorithm=hashes.SHA256(),
        length=32,
        salt=salt,
        info=b"clinical-platform:file-wrapping-key",
        backend=default_backend()
    ).derive(shared_secret)

# -----
# Hybrid encryption for files
# -----

def encrypt_file(plaintext: bytes, recipient_x25519_public) -> dict:
    """
    Encrypt file content with a random AES key.
    Then wrap that AES key for the recipient using X25519 + HKDF + AES-GCM.
    """
    file_key = os.urandom(32)
    encrypted_content = encrypt_data(file_key, plaintext)

    ephemeral_private, ephemeral_public = generate_x25519_keypair()
    shared_secret = derive_shared_secret(ephemeral_private,
    recipient_x25519_public)

    salt = ephemeral_public.public_bytes(
        serialization.Encoding.Raw,
        serialization.PublicFormat.Raw
    )
    wrapping_key = derive_file_wrapping_key(shared_secret, salt)
    wrapped_file_key = encrypt_data(wrapping_key, file_key)

    return {
        "ciphertext": encrypted_content["ciphertext"],
        "nonce": encrypted_content["nonce"],
        "wrapped_file_key": wrapped_file_key,
        "ephemeral_public": base64.b64encode(salt).decode()
    }

def decrypt_file(blob: dict, recipient_x25519_private) -> bytes:
    """Decrypt a hybrid-encrypted file blob."""
    from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PublicKey

    ephemeral_pub_bytes = base64.b64decode(blob["ephemeral_public"])
    ephemeral_public = X25519PublicKey.from_public_bytes(ephemeral_pub_bytes)

    shared_secret = derive_shared_secret(recipient_x25519_private,
    ephemeral_public)
    wrapping_key = derive_file_wrapping_key(shared_secret, ephemeral_pub_bytes)

    wrapped = blob["wrapped_file_key"]

```

```

    if isinstance(wrapped, str):
        wrapped = json.loads(wrapped)

    file_key = decrypt_data(wrapping_key, wrapped)

    return AESGCM(file_key).decrypt(
        base64.b64decode(blob["nonce"]),
        base64.b64decode(blob["ciphertext"]),
        None
    )

def wrap_file_for_recipient(file_key: bytes, recipient_x25519_public) -> dict:
    """Wrap an existing file key for a new recipient without re-encrypting
    content."""
    ephemeral_private, ephemeral_public = generate_x25519_keypair()
    shared_secret = derive_shared_secret(ephemeral_private,
    recipient_x25519_public)

    salt = ephemeral_public.public_bytes(
        serialization.Encoding.Raw,
        serialization.PublicFormat.Raw
    )
    wrapping_key = derive_file_wrapping_key(shared_secret, salt)
    wrapped = encrypt_data(wrapping_key, file_key)

    return {
        "wrapped_file_key": wrapped,
        "ephemeral_public": base64.b64encode(salt).decode()
    }

def unwrap_file_key(share_record: dict, recipient_x25519_private) -> bytes:
    """Unwrap a file key from a share record."""
    from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PublicKey

    ephemeral_pub_bytes = base64.b64decode(share_record["ephemeral_public"])
    ephemeral_public = X25519PublicKey.from_public_bytes(ephemeral_pub_bytes)

    shared_secret = derive_shared_secret(recipient_x25519_private,
    ephemeral_public)
    wrapping_key = derive_file_wrapping_key(shared_secret, ephemeral_pub_bytes)

    wrapped = share_record["wrapped_file_key"]
    if isinstance(wrapped, str):
        wrapped = json.loads(wrapped)

    return decrypt_data(wrapping_key, wrapped)

```

B.3.3 Serialisation Utilities

```

# -----
# Key serialisation helpers
# -----

```

```

def serialise_public_key(public_key) -> str:
    """Convert a public key object to PEM text."""
    return public_key.public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo
    ).decode()

def deserialise_public_key(pem_str: str):
    """Load a PEM public key string back into an object."""
    from cryptography.hazmat.primitives.serialization import load_pem_public_key
    return load_pem_public_key(pem_str.encode(), backend=default_backend())

def serialise_private_key(private_key) -> bytes:
    """Convert a private key object to PKCS8 PEM bytes."""
    return private_key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.NoEncryption()
    )

def deserialise_ed25519_private(pem_bytes: bytes):
    """Load an Ed25519 private key from PEM bytes."""
    from cryptography.hazmat.primitives.serialization import load_pem_private_key
    return load_pem_private_key(pem_bytes, password=None,
                                backend=default_backend())

def deserialise_x25519_private(pem_bytes: bytes):
    """Load an X25519 private key from PEM bytes."""
    from cryptography.hazmat.primitives.serialization import load_pem_private_key
    return load_pem_private_key(pem_bytes, password=None,
                                backend=default_backend())

def public_key_fingerprint_from_pem(pem_text: str) -> str:
    """Return a stable SHA-256 fingerprint of PEM text as base64."""
    digest = hashes.Hash(hashes.SHA256(), backend=default_backend())
    digest.update(pem_text.encode("utf-8"))
    return base64.b64encode(digest.finalize()).decode("utf-8")

```

B.4 Key Management

Lifecycle management of cryptographic identities, including Argon2id key derivation and password-based rewrapping.

B.4.1 KEK Derivation and Bundle Creation

```

# key_manager.py
# Advanced user key management for the clinical research platform.
# Responsibilities:
# 1. Derive a KEK from the user's password using Argon2id
# 2. Generate Ed25519 and X25519 keypairs
# 3. Encrypt private keys under the KEK before database storage
# 4. Support versioned keys so old data remains decryptable after rotation

```

```

# 5. Re-wrap all stored private keys when the user changes password

import os
import time
import json
import base64
from argon2.low_level import hash_secret_raw, Type

from crypto_core import (
    encrypt_data,
    decrypt_data,
    generate_ed25519_keypair,
    generate_x25519_keypair,
    serialise_public_key,
    serialise_private_key,
    deserialise_ed25519_private,
    deserialise_x25519_private,
    deserialise_public_key,
    public_key_fingerprint_from_pem
)
from storage import (
    get_connection,
    get_next_key_version,
    retire_active_keys,
    store_user_key_version,
    get_active_key_record,
    get_key_record,
    list_key_versions
)
from config import (
    ARGON2_TIME_COST,
    ARGON2_MEMORY_COST,
    ARGON2_PARALLELISM
)

def derive_kek(password: str, salt: bytes) -> bytes:
    """
    Derive a 256-bit Key Encryption Key from the user's password.

    Argon2id is used here because it is memory-hard and appropriate for
    password-based key derivation. The KEK is never stored directly.
    """
    return hash_secret_raw(
        secret=password.encode("utf-8"),
        salt=salt,
        time_cost=ARGON2_TIME_COST,
        memory_cost=ARGON2_MEMORY_COST,
        parallelism=ARGON2_PARALLELISM,
        hash_len=32,
        type=Type.ID
    )

```

```

def _build_key_bundle(password: str) -> dict:
    """
    Create one new logical key version for a user.

    Each version contains:
    - one Ed25519 pair for signing
    - one X25519 pair for key agreement
    - one KEK salt for deriving the wrapping key from the password

    The private keys are serialised to PEM and then encrypted with AES-GCM
    under the KEK. Public keys remain plaintext PEM so other users can use
    them for verification and secure sharing.
    """
    kek_salt = os.urandom(16)
    kek = derive_kek(password, kek_salt)

    ed_private, ed_public = generate_ed25519_keypair()
    x_private, x_public = generate_x25519_keypair()

    ed_private_pem = serialise_private_key(ed_private)
    x_private_pem = serialise_private_key(x_private)

    ed_private_enc = encrypt_data(kek, ed_private_pem)
    x_private_enc = encrypt_data(kek, x_private_pem)

    ed_public_pem = serialise_public_key(ed_public)
    x_public_pem = serialise_public_key(x_public)

    # Treat the two public keys as one logical identity version.
    combined_public_material = ed_public_pem + "\n" + x_public_pem
    key_fingerprint = public_key_fingerprint_from_pem(combined_public_material)

    return {
        "kek": kek,
        "kek_salt_b64": base64.b64encode(kek_salt).decode("utf-8"),
        "ed_private": ed_private,
        "ed_public": ed_public,
        "x_private": x_private,
        "x_public": x_public,
        "ed_private_enc": ed_private_enc,
        "x_private_enc": x_private_enc,
        "ed_public_pem": ed_public_pem,
        "x_public_pem": x_public_pem,
        "key_fingerprint": key_fingerprint
    }

def create_initial_keys(username: str, password: str) -> dict:
    """
    Create the first active key version for a new user.

```



```

This should only happen once, immediately after registration.
"""
key_version = get_next_key_version(username)
if key_version != 1:
    raise ValueError("Initial key creation expected key version 1.")

bundle = _build_key_bundle(password)
created_at = time.time()

store_user_key_version(
    username=username,
    key_version=key_version,
    status="active",
    x25519_private_enc=bundle["x_private_enc"],
    x25519_public=bundle["x_public_pem"],
    ed25519_private_enc=bundle["ed_private_enc"],
    ed25519_public=bundle["ed_public_pem"],
    key_fingerprint=bundle["key_fingerprint"],
    kek_salt=bundle["kek_salt_b64"],
    created_at=created_at,
    retired_at=None
)

bundle["username"] = username
bundle["key_version"] = key_version
bundle["status"] = "active"
return bundle

```

B.4.2 Key Versioning and Rotation

```

def rotate_user_keys(username: str, password: str) -> dict:
    """
    Rotate a user's keys by retiring the current active version and creating
    a brand new active version.

    Old versions remain in the database so historical content can still be
    verified and decrypted when needed.
    """
    retired_at = time.time()
    retire_active_keys(username, retired_at)

    key_version = get_next_key_version(username)
    bundle = _build_key_bundle(password)
    created_at = time.time()

    store_user_key_version(
        username=username,
        key_version=key_version,
        status="active",
        x25519_private_enc=bundle["x_private_enc"],
        x25519_public=bundle["x_public_pem"],
        ed25519_private_enc=bundle["ed_private_enc"],

```

```

        ed25519_public=bundle["ed_public_pem"],
        key_fingerprint=bundle["key_fingerprint"],
        kek_salt=bundle["kek_salt_b64"],
        created_at=created_at,
        retired_at=None
    )

    bundle["username"] = username
    bundle["key_version"] = key_version
    bundle["status"] = "active"
    return bundle

def load_active_user_keys(username: str, password: str) -> dict:
    """
    Load the currently active key version for a user.

    This is the normal function used after login because new operations should
    use the active keys, not retired ones.
    """
    record = get_active_key_record(username)
    if not record:
        raise ValueError(f"No active key record found for '{username}'.")

    return load_user_keys_by_version(username, password, record["key_version"])

def load_user_keys_by_version(username: str, password: str, key_version: int) -> dict:
    """
    Load one specific key version for a user.

    This is useful when:
    - decrypting or verifying historical material
    - supporting key rotation
    - debugging version-specific issues
    """
    record = get_key_record(username, key_version)
    if not record:
        raise ValueError(f"No key version {key_version} found for '{username}'.")

    kek_salt = base64.b64decode(record["kek_salt"])
    kek = derive_kek(password, kek_salt)

    ed_private_pem = decrypt_data(kek, record["ed25519_private_enc"])
    x_private_pem = decrypt_data(kek, record["x25519_private_enc"])

    return {
        "username": username,
        "key_version": record["key_version"],
        "status": record["status"],
        "key_fingerprint": record["key_fingerprint"],
        "kek": kek,
    }

```

```

    "ed_private": deserialise_ed25519_private(ed_private_pem),
    "ed_public": deserialise_public_key(record["ed25519_public"]),
    "x_private": deserialise_x25519_private(x_private_pem),
    "x_public": deserialise_public_key(record["x25519_public"]),
    "ed_public_pem": record["ed25519_public"],
    "x_public_pem": record["x25519_public"]
}

```

B.4.3 Credential-Driven Key Rewrapping

```

def rewrap_all_user_private_keys(username: str, old_password: str, new_password:
str) -> None:
    """
    Re-wrap every stored private key version under a new KEK.

    Important:
    This does not rotate the actual keypairs.
    It only changes the password-derived wrapping layer.

    This is what you want during a password change:
    - same public and private keys
    - same key versions
    - new KEK salt
    - new AES-GCM ciphertext for each stored private key
    """
    versions = list_key_versions(username)
    if not versions:
        raise ValueError(f"No key versions found for '{username}'.")

    conn = get_connection()

    try:
        for version in versions:
            record = get_key_record(username, version["key_version"])
            if not record:
                continue

            old_salt = base64.b64decode(record["kek_salt"])
            old_kek = derive_kek(old_password, old_salt)

            # First unwrap existing private keys using the old KEK.
            ed_private_pem = decrypt_data(old_kek, record["ed25519_private_enc"])
            x_private_pem = decrypt_data(old_kek, record["x25519_private_enc"])

            # Then derive a fresh KEK from the new password and a fresh salt.
            new_salt = os.urandom(16)
            new_kek = derive_kek(new_password, new_salt)

            new_ed_private_enc = encrypt_data(new_kek, ed_private_pem)
            new_x_private_enc = encrypt_data(new_kek, x_private_pem)

            conn.execute(

```

```

        """
        UPDATE user_key_versions
        SET ed25519_private_enc = ?,
            x25519_private_enc = ?,
            kek_salt = ?
        WHERE username = ? AND key_version = ?
        """
        (
            json.dumps(new_ed_private_enc),
            json.dumps(new_x_private_enc),
            base64.b64encode(new_salt).decode("utf-8"),
            username,
            record["key_version"]
        )

    conn.commit()

finally:
    conn.close()

```

B.5 Audit Logging

Implementation of the HMAC-chained, tamper-evident audit logging and verification logic.

```

# audit_log.py
# HMAC-SHA256 chained audit logging.

import os
import json
import hmac
import time
import hashlib

from storage import append_audit_entry, get_all_audit_entries, get_last_audit_hmac

HMAC_KEY_FILE = "audit_hmac.key"
GENESIS_PREV = "0" * 64

def _load_hmac_key() -> bytes:
    if os.path.exists(HMAC_KEY_FILE):
        if os.name != "nt":
            mode = os.stat(HMAC_KEY_FILE).st_mode & 0o777
            if mode & 0o077:
                raise PermissionError(
                    "audit_hmac.key permissions are too broad; expected owner-only
access."
                )
        with open(HMAC_KEY_FILE, "rb") as f:
            return f.read()

```

```

key = os.urandom(32)
fd = os.open(HMAC_KEY_FILE, os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o600)
with os.fdopen(fd, "wb") as f:
    f.write(key)
return key

def _compute_hmac(key: bytes, prev_hmac: str, entry_json: str) -> str:
    msg = (prev_hmac + entry_json).encode("utf-8")
    return hmac.new(key, msg, hashlib.sha256).hexdigest()

def sanitise_audit_detail(action: str, detail: str) -> str:
    if not detail:
        return ""

    allowed_prefixes = {
        "USER_REGISTERED": {"user=", "role="},
        "LOGIN_SUCCESS": {"MFA"},
        "LOGIN_BLOCKED": {"locked_for="},
        "UPLOAD_FINDING": {"file_id="},
        "VIEW_FINDING": {"file_id="},
        "VIEW_SHARED_FINDING": {"file_id="},
        "SHARE_FINDING": {"file_id=", "with="},
        "UPLOAD_DATASET": {"file_id=", "subject_ref="},
        "VIEW_DATASET": {"file_id="},
        "VIEW_SHARED_DATASET": {"file_id="},
        "SHARE_DATASET": {"file_id=", "with="},
        "ERASE_DATASET": {"file_id="},
        "DECRYPT_FAIL": {"file_id=", "error="},
        "UNWRAP_FAIL": {"file_id=", "error="},
        "RESHARE_DENIED": {"file_id="},
        "VERIFY_LOG_INTEGRITY": {"valid="},
        "VERIFY_SIGNATURE": {"file_id=", "valid="},
        "VIEW_AUDIT_LOG": {"entries"},
    }

    prefixes = allowed_prefixes.get(action)
    if prefixes is None:
        return ""

    tokens = detail.split()
    safe_tokens = []

    for token in tokens:
        if token == "MFA" and "MFA" in prefixes:
            safe_tokens.append(token)
            continue
        if token == "verified" and "MFA" in prefixes:
            safe_tokens.append(token)
            continue
        if token == "entries" and "entries" in prefixes:
            safe_tokens.append(token)

```

```

        continue

    for prefix in prefixes:
        if token.startswith(prefix):
            safe_tokens.append(token)
            break

    return " ".join(safe_tokens)

def log_event(user: str, role: str, action: str, detail: str = "") -> None:
    key = _load_hmac_key()
    prev_hmac = get_last_audit_hmac() or GENESIS_PREV
    safe_detail = sanitise_audit_detail(action, detail)

    entry_data = json.dumps({
        "timestamp": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
        "user": user,
        "role": role,
        "action": action,
        "detail": safe_detail
    }, separators=(",", ":"))

    entry_hmac = _compute_hmac(key, prev_hmac, entry_data)
    append_audit_entry(entry_data, entry_hmac)

def verify_log_integrity() -> tuple:
    key = _load_hmac_key()
    entries = get_all_audit_entries()

    tampered = []
    prev_digest = GENESIS_PREV

    for i, entry in enumerate(entries):
        expected = _compute_hmac(key, prev_digest, entry["data"])
        if not hmac.compare_digest(expected, entry["hmac"]):
            tampered.append(i)
            prev_digest = entry["hmac"]

    return (len(tampered) == 0, tampered)

def read_log() -> list:
    return [json.loads(e["data"]) for e in get_all_audit_entries()]

```

B.6 Storage and Configuration

System configurations and SQLite repository abstractions for persisting platform state.

B.6.1 System Constants

```

# config.py
# System-wide constants for the platform.

DB_FILE = "clinical_platform.db"

```

```

SESSION_TIMEOUT_SECONDS = 900    # 15 minutes
MAX_FAILED_ATTEMPTS = 3
LOCKOUT_DURATION = 300           # 5 minutes in seconds

TOTP_ISSUER = "ClinicalResearchPlatform"

# Argon2id parameters (OWASP 2023 / RFC 9106 style settings).
ARGON2_TIME_COST = 3
ARGON2_MEMORY_COST = 65536
ARGON2_PARALLELISM = 4

```

B.6.2 Database Initialisation and Schemas

```

# storage.py
# SQLite operations for all persistent state.

import sqlite3
import json
import time
from config import DB_FILE

def get_connection():
    conn = sqlite3.connect(DB_FILE)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON")
    return conn

def initialise_db():
    conn = get_connection()
    c = conn.cursor()

    c.executescript("""
CREATE TABLE IF NOT EXISTS users (
    username TEXT PRIMARY KEY,
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL,
    first_name TEXT,
    last_name TEXT,
    location TEXT,
    totp_secret TEXT NOT NULL,
    failed_attempts INTEGER DEFAULT 0,
    locked_until REAL DEFAULT 0,
    account_status TEXT DEFAULT 'active',
    password_changed_at REAL,
    mfa_reset_at REAL,
    created_at REAL NOT NULL
);

CREATE TABLE IF NOT EXISTS user_key_versions (

```

```

        username TEXT NOT NULL,
        key_version INTEGER NOT NULL,
        status TEXT NOT NULL,
        x25519_private_enc TEXT NOT NULL,
        x25519_public TEXT NOT NULL,
        ed25519_private_enc TEXT NOT NULL,
        ed25519_public TEXT NOT NULL,
        key_fingerprint TEXT NOT NULL,
        kek_salt TEXT NOT NULL,
        created_at REAL NOT NULL,
        retired_at REAL,
        PRIMARY KEY (username, key_version),
        FOREIGN KEY (username) REFERENCES users(username)
    );

CREATE TABLE IF NOT EXISTS files (
    file_id TEXT PRIMARY KEY,
    owner TEXT NOT NULL,
    role TEXT NOT NULL,
    filename TEXT NOT NULL,
    ciphertext TEXT NOT NULL,
    nonce TEXT NOT NULL,
    signature TEXT NOT NULL,
    content_hash TEXT NOT NULL DEFAULT '',
    ed25519_public TEXT NOT NULL,
    created_at REAL NOT NULL,
    FOREIGN KEY (owner) REFERENCES users(username)
);

CREATE TABLE IF NOT EXISTS shares (
    share_id TEXT PRIMARY KEY,
    file_id TEXT NOT NULL,
    shared_by TEXT NOT NULL,
    shared_with TEXT NOT NULL,
    wrapped_file_key TEXT NOT NULL,
    ephemeral_public TEXT NOT NULL,
    created_at REAL NOT NULL,
    FOREIGN KEY (file_id) REFERENCES files(file_id)
);

CREATE TABLE IF NOT EXISTS audit_log (
    entry_id INTEGER PRIMARY KEY AUTOINCREMENT,
    data TEXT NOT NULL,
    hmac TEXT NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_user_key_versions_user
ON user_key_versions(username);

CREATE INDEX IF NOT EXISTS idx_user_key_versions_status
ON user_key_versions(username, status);

```



```

CREATE INDEX IF NOT EXISTS idx_files_owner
ON files(owner);

CREATE INDEX IF NOT EXISTS idx_files_role
ON files(role);

CREATE INDEX IF NOT EXISTS idx_shares_shared_with
ON shares(shared_with);

CREATE INDEX IF NOT EXISTS idx_shares_shared_by
ON shares(shared_by);

CREATE INDEX IF NOT EXISTS idx_shares_file_id
ON shares(file_id);
""")

existing_cols = [row["name"] for row in c.execute("PRAGMA
table_info(users)").fetchall()]
migrations = {
    "first_name": "ALTER TABLE users ADD COLUMN first_name TEXT",
    "last_name": "ALTER TABLE users ADD COLUMN last_name TEXT",
    "location": "ALTER TABLE users ADD COLUMN location TEXT",
    "account_status": "ALTER TABLE users ADD COLUMN account_status TEXT
DEFAULT 'active'",
    "password_changed_at": "ALTER TABLE users ADD COLUMN password_changed_at
REAL",
    "mfa_reset_at": "ALTER TABLE users ADD COLUMN mfa_reset_at REAL",
}
for col, stmt in migrations.items():
    if col not in existing_cols:
        c.execute(stmt)

file_cols = [row["name"] for row in c.execute("PRAGMA
table_info(files)").fetchall()]
file_migrations = {
    "content_hash": "ALTER TABLE files ADD COLUMN content_hash TEXT NOT NULL
DEFAULT ''",
}
for col, stmt in file_migrations.items():
    if col not in file_cols:
        c.execute(stmt)

conn.commit()
conn.close()

```

B.6.3 User and Key Entity Persistence

```

def create_user(username, password_hash, role, totp_secret, created_at,
first_name=None, last_name=None, location=None):
    conn = get_connection()
    conn.execute(

```

```

"""
INSERT INTO users (
    username, password_hash, role, first_name, last_name, location,
    totp_secret, failed_attempts, locked_until, account_status,
    password_changed_at, mfa_reset_at, created_at
)
VALUES (?, ?, ?, ?, ?, ?, ?, 0, 0, 'active', ?, NULL, ?)
"""
(
    username,
    password_hash,
    role,
    first_name,
    last_name,
    location,
    totp_secret,
    created_at,
    created_at,
)
)
conn.commit()
conn.close()

def get_user(username):
    conn = get_connection()
    row = conn.execute("SELECT * FROM users WHERE username=?",
(username,)).fetchone()
    conn.close()
    return dict(row) if row else None

def update_failed_attempts(username, attempts, locked_until=0):
    conn = get_connection()
    conn.execute(
        "UPDATE users SET failed_attempts=?, locked_until=? WHERE username=?",
        (attempts, locked_until, username)
    )
    conn.commit()
    conn.close()

def reset_failed_attempts(username):
    conn = get_connection()
    conn.execute(
        "UPDATE users SET failed_attempts=0, locked_until=0 WHERE username=?",
        (username,)
    )
    conn.commit()
    conn.close()

def update_password_hash(username, password_hash):
    conn = get_connection()
    conn.execute(

```

```

        "UPDATE users SET password_hash=?, password_changed_at=? WHERE
username=?",
        (password_hash, time.time(), username)
    )
    conn.commit()
    conn.close()

def store_user_key_version(
    username,
    key_version,
    status,
    x25519_private_enc,
    x25519_public,
    ed25519_private_enc,
    ed25519_public,
    key_fingerprint,
    kek_salt,
    created_at,
    retired_at=None,
):
    conn = get_connection()
    conn.execute(
        """
        INSERT INTO user_key_versions (
            username, key_version, status, x25519_private_enc, x25519_public,
            ed25519_private_enc, ed25519_public, key_fingerprint,
            kek_salt, created_at, retired_at
        )
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        """,
        (
            username,
            key_version,
            status,
            json.dumps(x25519_private_enc),
            x25519_public,
            json.dumps(ed25519_private_enc),
            ed25519_public,
            key_fingerprint,
            kek_salt,
            created_at,
            retired_at,
        )
    )
    conn.commit()
    conn.close()

def get_next_key_version(username):
    conn = get_connection()
    row = conn.execute(
        """

```

```

        SELECT COALESCE(MAX(key_version), 0) AS max_version
        FROM user_key_versions
        WHERE username=?
        """
        (username,)
    ).fetchone()
    conn.close()
    return int(row["max_version"]) + 1

def retire_active_keys(username, retired_at):
    conn = get_connection()
    conn.execute(
        """
        UPDATE user_key_versions
        SET status='retired', retired_at=?
        WHERE username=? AND status='active'
        """,
        (retired_at, username)
    )
    conn.commit()
    conn.close()

def get_active_key_record(username):
    conn = get_connection()
    row = conn.execute(
        """
        SELECT *
        FROM user_key_versions
        WHERE username=? AND status='active'
        ORDER BY key_version DESC
        LIMIT 1
        """,
        (username,)
    ).fetchone()
    conn.close()

    if not row:
        return None

    record = dict(row)
    record["x25519_private_enc"] = json.loads(record["x25519_private_enc"])
    record["ed25519_private_enc"] = json.loads(record["ed25519_private_enc"])
    return record

def get_key_record(username, key_version):
    conn = get_connection()
    row = conn.execute(
        "SELECT * FROM user_key_versions WHERE username=? AND key_version=?",
        (username, key_version)
    ).fetchone()

```

```

conn.close()

if not row:
    return None

record = dict(row)
record["x25519_private_enc"] = json.loads(record["x25519_private_enc"])
record["ed25519_private_enc"] = json.loads(record["ed25519_private_enc"])
return record

def list_key_versions(username):
    conn = get_connection()
    rows = conn.execute(
        """
        SELECT username, key_version, status, key_fingerprint, created_at,
retired_at
        FROM user_key_versions
        WHERE username=?
        ORDER BY key_version DESC
        """,
        (username,)
    ).fetchall()
    conn.close()
    return [dict(r) for r in rows]

def get_user_active_public_keys(username):
    conn = get_connection()
    row = conn.execute(
        """
        SELECT username, key_version, x25519_public, ed25519_public,
key_fingerprint, status
        FROM user_key_versions
        WHERE username=? AND status='active'
        ORDER BY key_version DESC
        LIMIT 1
        """,
        (username,)
    ).fetchone()
    conn.close()
    return dict(row) if row else None

def update_user_status(username, status):
    conn = get_connection()
    conn.execute(
        "UPDATE users SET account_status=? WHERE username=?",
        (status, username)
    )
    conn.commit()
    conn.close()

def list_users():

```

```

conn = get_connection()
rows = conn.execute(
    """
    SELECT username, role, first_name, last_name, location,
           account_status, created_at, password_changed_at
    FROM users
    ORDER BY created_at ASC
    """
).fetchall()
conn.close()
return [dict(r) for r in rows]

```

B.6.4 File, Share, and Audit Persistence

```

def store_file(file_id, owner, role, filename, ciphertext, nonce, signature,
               content_hash, ed25519_public, created_at):
    conn = get_connection()
    conn.execute(
        """
        INSERT INTO files (
            file_id, owner, role, filename, ciphertext,
            nonce, signature, content_hash, ed25519_public, created_at
        )
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        """,
        (
            file_id,
            owner,
            role,
            filename,
            ciphertext,
            nonce,
            signature,
            content_hash,
            ed25519_public,
            created_at,
        )
    )
    conn.commit()
    conn.close()

def get_files_for_role(role):
    conn = get_connection()
    rows = conn.execute(
        "SELECT * FROM files WHERE role=? ORDER BY created_at DESC",
        (role,)
    ).fetchall()
    conn.close()
    return [dict(r) for r in rows]

```

```

def get_file(file_id):
    conn = get_connection()
    row = conn.execute("SELECT * FROM files WHERE file_id=?",
(file_id,)).fetchone()
    conn.close()
    return dict(row) if row else None

def delete_file(file_id):
    conn = get_connection()
    conn.execute("DELETE FROM shares WHERE file_id=?", (file_id,))
    conn.execute("DELETE FROM files WHERE file_id=?", (file_id,))
    conn.commit()
    conn.close()

def store_share(share_id, file_id, shared_by, shared_with, wrapped_file_key,
ephemeral_public, created_at):
    conn = get_connection()
    conn.execute(
        """
        INSERT INTO shares (
            share_id, file_id, shared_by, shared_with,
            wrapped_file_key, ephemeral_public, created_at
        )
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)
        """,
        (
            share_id,
            file_id,
            shared_by,
            shared_with,
            json.dumps(wrapped_file_key),
            ephemeral_public,
            created_at,
        )
    )
    conn.commit()
    conn.close()

def get_shares_for_user(username):
    conn = get_connection()
    rows = conn.execute(
        """
        SELECT
            s.*, f.filename, f.ciphertext, f.nonce, f.signature,
            f.content_hash, f.ed25519_public, f.owner, f.role
        FROM shares s
        JOIN files f ON s.file_id = f.file_id
        WHERE s.shared_with=?
        ORDER BY s.created_at DESC
        """,
        (username,)
    )

```

```

        ).fetchall()
        conn.close()
        return [dict(r) for r in rows]

def get_shares_by_user(username):
    conn = get_connection()
    rows = conn.execute(
        "SELECT * FROM shares WHERE shared_by=? ORDER BY created_at DESC",
        (username,)
    ).fetchall()
    conn.close()
    return [dict(r) for r in rows]

def append_audit_entry(data: str, hmac_value: str):
    conn = get_connection()
    conn.execute("INSERT INTO audit_log (data, hmac) VALUES (?, ?)", (data,
hmac_value))
    conn.commit()
    conn.close()

def get_all_audit_entries():
    conn = get_connection()
    rows = conn.execute("SELECT data, hmac FROM audit_log ORDER BY
entry_id").fetchall()
    conn.close()
    return [dict(r) for r in rows]

def get_last_audit_hmac():
    conn = get_connection()
    row = conn.execute("SELECT hmac FROM audit_log ORDER BY entry_id DESC LIMIT
1").fetchone()
    conn.close()
    return row["hmac"] if row else "0" * 64

```

B.7 Main Application Flow and Administrator Functions

Administrative management controls, disaster recovery backup generation, and the primary system execution loop.

B.7.1 Administrator Controls and System Backup

```

def export_backup(username: str, role: str) -> None:
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    backup_root = "backups"
    os.makedirs(backup_root, exist_ok=True)

    export_dir = os.path.join(backup_root, f"{username}_{timestamp}")
    os.makedirs(export_dir, exist_ok=True)

    copied_any = False

```



```

    for folder in ["keys", "data", "logs"]:
        if os.path.isdir(folder):
            shutil.copytree(folder, os.path.join(export_dir, folder),
dirs_exist_ok=True)
            copied_any = True

    for filename in os.listdir("."):
        if filename.endswith(".db") or filename.endswith(".sqlite") or
filename.endswith(".sqlite3"):
            shutil.copy2(filename, os.path.join(export_dir, filename))
            copied_any = True

    archive_base = os.path.join(backup_root, f"{username}_{timestamp}")
    archive_path = shutil.make_archive(archive_base, "zip", export_dir)

    if copied_any:
        log_event(username, role, "BACKUP_EXPORT", f"path={archive_path}")
        print(colour(f"[OK] Backup created: {archive_path}", GREEN))
    else:
        print(colour("[!] No backup source files were found.", RED))

def admin_menu(username: str, keys: dict, session_start: float, password: str) ->
None:
    role = "admin"

    while True:
        if not check_session(session_start):
            print(colour("[!] Session expired. Please log in again.", RED))
            log_event(username, role, "SESSION_TIMEOUT", "")
            break

        print("\n--- Admin Menu ---")
        print("1 List users")
        print("2 Create new user")
        print("3 Disable user")
        print("4 Enable user")
        print("5 Export system backup")
        print("6 Change password")
        print("7 Logout")
        choice = input("Choice: ").strip()

        if choice == "1":
            users = list_users()

            if not users:
                print("No users found.")
                continue

            print(f"\n{'Username':<18} {'Role':<12} {'Status':<10}
{'Name':<24}{'Location':<18}{'Pwd Changed'}")

```

```

        print("-" * 80)
        for u in users:
            full_name = f"{u.get('first_name', '')} {u.get('last_name',
            '')}}".strip()
            pwd_changed_text = format_timestamp(u.get("password_changed_at"))
            print(
                f"{u['username']:<18} "
                f"{u['role']:<12} "
                f"{u.get('account_status', 'active'):<10} "
                f"{full_name:<24} "
                f"{u.get('location', ''):<18}"
                f"{pwd_changed_text}"
            )

            log_event(username, role, "LIST_USERS", "")

    elif choice == "2":
        user_creation_flow(created_by=username, creator_role=role,
        admin_mode=True)

    elif choice == "3":
        target = input("Username to disable: ").strip()

        if target == username:
            print(colour("[!] You cannot disable your own admin account.",
            RED))
            continue

        target_user = get_user(target)
        if not target_user:
            print(colour("[!] User not found.", RED))
            continue

        if target_user.get("account_status") != "active":
            print(colour("[!] User is already not active.", RED))
            continue

        update_user_status(target, "inactive")
        print(colour(f"[OK] User '{target}' has been disabled.", GREEN))
        log_event(username, role, "DISABLE_USER", f"user={target}")

    elif choice == "4":
        target = input("Username to enable: ").strip()

        target_user = get_user(target)
        if not target_user:
            print(colour("[!] User not found.", RED))
            continue

        if target_user.get("account_status") == "active":
            print(colour("[!] User is already active.", RED))

```

```

        continue

        update_user_status(target, "active")
        print(colour(f"[OK] User '{target}' has been enabled.", GREEN))
        log_event(username, role, "ENABLE_USER", f"user={target}")

    elif choice == "5":
        export_backup(username, role)

    elif choice == "6":
        changed = password_change_flow(username, role)
        if changed:
            break
    elif choice == "7":
        log_event(username, role, "LOGOUT", "")
        print("Logged out.")
        break
    else:
        print(colour("[!] Unrecognised option.", RED))

```

B.7.2 Application Entry Point and Setup Bootstrapping

```

# main.py
# Entry point, registration, login, MFA, and session management.
import os
import shutil
import time
import getpass
import uuid
import json
import pyotp
import hashlib
from cryptography.exceptions import InvalidTag
from datetime import datetime
from config import SESSION_TIMEOUT_SECONDS, TOTP_ISSUER
from storage import (
    initialise_db,
    get_user,
    get_files_for_role,
    get_file,
    delete_file,
    store_file,
    store_share,
    get_shares_for_user,
    get_user_active_public_keys,
    list_users,
    update_user_status,
)
from auth import authenticate, register_user, verify_totp, change_password
from audit_log import log_event, verify_log_integrity, read_log
from key_manager import load_active_user_keys, rotate_user_keys
from crypto_core import (

```

```

    encrypt_file,
    decrypt_file,
    wrap_file_for_recipient,
    unwrap_file_key,
    sign_data,
    verify_signature,
    serialise_public_key,
    deserialise_public_key,
)

PLATFORM_NAME = "Aurelia Clinical Research Platform"
PLATFORM_TAGLINE = "Cross-Border Clinical Research Collaboration"
OTP_DISPLAY_SECONDS = 90

GREEN = "\033[92m"
RED = "\033[91m"
YELLOW = "\033[93m"
CYAN = "\033[96m"
BOLD = "\033[1m"
RESET = "\033[0m"

def colour(text: str, code: str) -> str:
    return f"{code}{text}{RESET}"

def check_session(session_start: float) -> bool:
    return (time.time() - session_start) < SESSION_TIMEOUT_SECONDS

def display_current_otp(user: dict, username: str) -> None:
    current_otp = pyotp.TOTP(user["totp_secret"]).now()
    print(colour(f"[DEMO] Current MFA code for {username}: {current_otp}",
YELLOW))

def format_timestamp(ts):
    if not ts:
        return "Never"
    return datetime.fromtimestamp(ts).strftime("%Y-%m-%d %H:%M")

def self_key_rotation_flow(username: str, role: str, password: str) -> bool:
    print("\n--- Rotate My Cryptographic Keys ---")
    print("This will create a new active signing and encryption key version.")
    print("Previous key versions will remain stored for historical verification.")
    confirm = input("Type ROTATE to continue: ").strip()

    if confirm != "ROTATE":
        print(colour("[!] Key rotation cancelled.", YELLOW))
        return False

    try:
        new_keys = rotate_user_keys(username, password)
        log_event(

```

```

        username,
        role,
        "KEY_ROTATED",
        f"self new_version={new_keys['key_version']}"
fingerprint={new_keys['key_fingerprint']}"
    )
    print(colour(
        f"[OK] Keys rotated successfully. New active version:
{new_keys['key_version']}",
        GREEN
    ))
    print(colour("[!] Please log in again to continue with the new active
keys.", YELLOW))
    return True
except Exception as e:
    print(colour(f"[!] Key rotation failed: {e}", RED))
    log_event(username, role, "KEY_ROTATION_FAIL", f"self
error={type(e).__name__}")
    return False

def password_change_flow(username: str, role: str) -> bool:
    print("\n--- Change Password ---")
    current_password = getpass.getpass("Current password: ")
    new_password = getpass.getpass("New password: ")
    confirm_password = getpass.getpass("Confirm new password: ")

    if new_password != confirm_password:
        print(colour("! New passwords do not match.", RED))
        log_event(username, role, "PASSWORD_CHANGE_FAIL", "confirmation_mismatch")
        return False

    if current_password == new_password:
        print(colour("! New password must be different from the current
password.", RED))
        log_event(username, role, "PASSWORD_CHANGE_FAIL", "password_reuse")
        return False

    try:
        change_password(username, current_password, new_password)
        print(colour("OK Password changed successfully. Please log in again.",
GREEN))
        log_event(username, role, "PASSWORD_CHANGED", "")
        return True
    except Exception as e:
        print(colour(f"! Password change failed: {e}", RED))
        log_event(username, role, "PASSWORD_CHANGE_FAIL", type(e).__name__)
        return False

def user_creation_flow(created_by=None, creator_role=None, admin_mode=False):
    print("\n--- Create New User ---" if admin_mode else "\n--- Register New
Account ---")

```

```

first_name = input("First name      : ").strip()
last_name = input("Last name       : ").strip()
location = input("Location         : ").strip()
new_username = input("Username      : ").strip()

print("Select role:")
print("  1  Researcher")
print("  2  Clinician")
print("  3  Auditor")
role_map = {"1": "researcher", "2": "clinician", "3": "auditor"}
role_choice = input("Role: ").strip()
new_role = role_map.get(role_choice)

if not new_role:
    print(colour("[!] Invalid role.", RED))
    return

if get_user(new_username):
    print(colour("[!] Username already exists.", RED))
    return

print(colour(
    "Password requirements: at least 8 characters, one uppercase letter, one
lowercase letter, one digit, and one special character.",
    YELLOW
))

new_password = getpass.getpass("Temporary password : " if admin_mode else
"Password      : ")
confirm_password = getpass.getpass("Confirm password : ")

if new_password != confirm_password:
    print(colour("[!] Passwords do not match.", RED))
    return

try:
    totp_secret = register_user(
        username=new_username,
        password=new_password,
        role=new_role,
        first_name=first_name,
        last_name=last_name,
        location=location,
    )

    uri = pyotp.TOTP(totp_secret).provisioning_uri(
        name=new_username,
        issuer_name=TOTP_ISSUER,
    )

```

```

print(colour(f"[OK] User created: {new_username} ({new_role})", GREEN))
print(colour(f"[!] TOTP Secret: {totp_secret}", YELLOW))
print(colour("[!] Add to authenticator app:", YELLOW))
print(f"    {uri}")

if admin_mode:
    log_event(
        created_by,
        creator_role,
        "ADMIN_CREATE_USER",
        f"user={new_username} role={new_role}"
    )
else:
    log_event(
        "SYSTEM",
        "admin",
        "USER_REGISTERED",
        f"user={new_username} role={new_role}"
    )

except ValueError as e:
    print(colour(f"[!] {e}", RED))

def main():
    initialise_db()

    if not get_user("admin"):
        print(colour("[SETUP] No admin account found. Create the built-in admin
now.", YELLOW))
        while True:
            admin_password = getpass.getpass("Set admin password : ")
            confirm_password = getpass.getpass("Confirm admin password : ")

            if admin_password != confirm_password:
                print(colour("[!] Passwords do not match.", RED))
                continue

            try:
                admin_totp = register_user(
                    username="admin",
                    password=admin_password,
                    role="admin",
                    first_name="System",
                    last_name="Administrator",
                    location="Internal",
                )
                log_event("SYSTEM", "admin", "BOOTSTRAP_ADMIN_CREATED",
"user=admin")
                print(colour("[OK] Built-in admin account created.", GREEN))
                print(colour(f"[!] Admin TOTP secret: {admin_totp}", YELLOW))
                break

```

```

        except ValueError as e:
            print(colour(f"[!] {e}", RED))

print("=" * 60)
print(colour(f" {PLATFORM_NAME}", BOLD + CYAN))
print(f" {PLATFORM_TAGLINE}")
print("=" * 60)

while True:
    print("\n 1 Login")
    print(" 2 Register new account")
    print(" 3 Exit")
    opt = input("Option: ").strip()

    if opt == "3":
        print("Goodbye.")
        break

    if opt == "2":
        user_creation_flow()
        continue

    if opt != "1":
        print(colour(f"[!] Unrecognised option.", RED))
        continue

    username = input("Username : ").strip()
    password = getpass.getpass("Password : ")
    result = authenticate(username, password)

    if result is None:
        print(colour(f"[!] Login failed. Check your credentials.", RED))
        log_event(username, "unknown", "LOGIN_FAIL", "")
        continue

    if isinstance(result, str) and result.startswith("locked:"):
        remaining = result.split(":")[1]
        print(colour(f"[!] Account locked. Try again in {remaining} seconds.",
RED))
        log_event(username, "unknown", "LOGIN_BLOCKED",
f"locked_for={remaining}s")
        continue

    user = result
    role = user["role"]

    display_current_otp(user, username)

    totp_token = input("Authenticator code : ").strip()
    if not verify_totp(username, totp_token := totp_token):
        print(colour(f"[!] Invalid authenticator code.", RED))

```



```

        log_event(username, role, "MFA_FAIL", "")
        continue

    try:
        keys = load_active_user_keys(username, password)
    except Exception as e:
        print(colour(f"[!] Key loading failed: {e}", RED))
        log_event(username, role, "KEY_LOAD_FAIL", type(e).__name__)
        continue

    session_start = time.time()
    display_name = f"{user.get('first_name', '')} {user.get('last_name',
'' )}".strip() or username

    print(colour(f"\n[+] Welcome, {display_name}", GREEN))
    print(f"Role      : {role}")
    print(f"Location : {user.get('location', 'Not set')}")

    log_event(username, role, "LOGIN_SUCCESS", "mfa_ok")

    if role == "researcher":
        researcher_menu(username, keys, session_start, password)
    elif role == "clinician":
        clinician_menu(username, keys, session_start, password)
    elif role == "auditor":
        auditor_menu(username, keys, session_start)
    elif role == "admin":
        admin_menu(username, keys, session_start, password)
    else:
        print(colour("[!] Unrecognised role.", RED))
        log_event(username, role, "ACCESS_DENIED", "")

if __name__ == "__main__":
    main()

```